Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Multi-Objective Optimization within the Pareto Set

Applications to Model Merging

Bachelor's Thesis

Silvan Bolt

March 18, 2025

Advisors: Prof. Dr. Fanny Yang, Tobias Wegel
Department of Computer Science, ETH Zürich

Abstract

Most simple machine learning methods are based on optimizing the parameters of a model with respect to one objective function, for example, by using gradient descent. But many modern machine learning applications require multi-objective optimization, which means finding an optimum with respect to several objective functions that may conflict with one another.

Finding optimal model parameters requires balancing competing objectives. To address this trade-off, we shift the algorithmic goal toward achieving Pareto-optimality, where improving one objective may come at the cost of another. The set of such Pareto-optimal points is called the Pareto set, which is often infinite. Numerous algorithms exist that take an initial point and iteratively refine it toward a Pareto-optimal solution, ensuring convergence to a point in the Pareto set. This approach is known as multi-objective optimization toward the Pareto set. Since multiple Pareto-optimal solutions exist, traditional algorithms optimizing toward the Pareto set often select a solution arbitrarily.

However, in some cases, after reaching a Pareto-optimal solution, the user may wish to prioritize or refocus on a subset of the objective functions —adjusting the trade-off between them. For instance, consider the two objective functions given by the prediction risk and the unfairness score, both of which are ideally minimized. After obtaining a Pareto-optimal model, the user might decide to shift the balance between these objectives, prioritizing one over the other. In other cases, the user may wish to minimize a criterion function, constrained to solutions in the Pareto set, which inherently alters the trade-offs between the original objectives.

One approach to this problem is to restart the optimization process to find a new trade-off. However, this can lead to significant computational overhead, as it does not leverage the fact that one already has a Pareto-optimal model. Moreover, it does not necessarily guarantee convergence to the desired solution.

Motivated by this limitation, a recent line of research has introduced *Optimization in the Pareto set*, where an additional criterion function is optimized within the Pareto set. This function encodes the user's preference.

An example where this is particularly relevant is in model merging, a process where the user is combining multiple pre-trained models into one model, trading off the abilities of the individual models, ideally leading to better generalization downstream.

In this thesis, we investigate various algorithms for optimization toward and within the Pareto set and compare these algorithms across different example settings. For PNGD and PMM, two algorithms that operate within the Pareto set, we analyze PNGD's convergence behavior and PMM's runtime as a function of feature size. Finally, we

demonstrate a practical application of optimization within the Pareto set in the context of model merging.

Contents

Contents iii

1 Introduction 1
1.1 Multi-Objective Optimization 1
1.2 Open Problems 3
1.2.1 Optimization toward the Pareto set 3
1.2.2 Optimization within the Pareto set 4
1.2.3 Model Merging 5
1.3 Research Question 6

2 Preliminaries of Multi-Objective Optimization 7
2.1 Definitions & Preliminaries 7
2.2 Optimization toward the Pareto set 10
2.2.1 Scalarization Technique 10
2.2.2 Gradient-Based Algorithms 10
2.2.3 Bilevel Optimization Algorithms 11
2.2.4 Representative Subsample Algorithms 11
2.3 Optimization within the Pareto set 12
2.4 Model Merging 14
2.5 Open Problems 15

3 Related Work 19
3.1 Optimization toward the Pareto set 19
3.1.1 Representative Subsampling 19
3.1.2 Gradient-Based Algorithms 20
3.1.3 Bilevel Optimization 23
3.2 Optimization within the Pareto set 24
3.2.1 Pareto Navigation Gradient Descent 24
3.2.2 Pareto Majorization-Minimization 25
3.3 Model Merging 27

4 Comparison & Building Blocks **29**
4.1 Descent Direction 29
4.2 Pareto Stationarity 32
4.3 Optimization within the Pareto Set 32
4.4 Outcomes 32

5 Experiments **35**
5.1 Experimental Setup 35
5.1.1 Optimization toward the Pareto Set 35
5.1.2 Example of Non-Convergence for PNGD 37
5.1.3 Runtime of PMM 38
5.1.4 Optimization within the Pareto set for Model Merging 39
5.2 Results 41
5.2.1 Optimization toward the Pareto Set 41
5.2.2 Example of Non-Convergence for PNGD 43
5.2.3 Runtime of PMM 43
5.2.4 Optimization in the Pareto set for Model Merging . . . 43

6 Conclusion **55**

Bibliography **57**

Chapter 1

Introduction

1.1 Multi-Objective Optimization

In single-objective optimization, the goal is to optimize a single objective function, which can be achieved without conflicts. However, in multi-objective optimization (MOO), multiple competing objectives must be considered simultaneously. These objectives are often dependent, meaning that improving one may come at the expense of another. Instead of a single optimal solution, MOO yields a set of trade-off solutions, known as the *Pareto set*, where no objective can be improved without worsening at least one other. This allows decision-makers to select solutions based on their specific priorities.

Example 1 —Car Purchase

To better illustrate multi-objective optimization, let us consider a real-world example with two conflicting objectives, where one seeks to minimize costs while the other aims to maximize comfort.

When purchasing a car, buyers often face a trade-off between the following objectives:

- *Minimize Cost:* Reducing the purchase price, fuel consumption, and maintenance expenses.
- *Maximize Comfort:* Enhancing the driving experience with premium features such as leather seats, noise insulation and high-end infotainment systems.

A budget-friendly car saves money but may lack comfort and advanced features, whereas a luxury car offers greater comfort but comes with higher upfront and maintenance costs. Buyers make their decision by balancing financial constraints and driving comfort.

Example 2 —Drone Delivery

Let us consider an example with three objectives. A practical example could be drone delivery route optimization, where logistics companies must balance multiple competing objectives to improve efficiency. The key objectives in this scenario are:

- *Minimize Delivery Time:* Ensuring drones reach their destinations as quickly as possible to meet customer expectations.
- *Minimize Energy Consumption:* Reducing battery usage per trip extends drone range and lowers operational costs.
- *Maximize Package Safety:* Avoiding high-wind areas, turbulence, or unsafe landing zones ensures packages arrive undamaged.

These objectives often conflict. For instance, taking the shortest route minimizes delivery time but may require flying through turbulent regions, increasing energy consumption and risking package damage. Choosing safer routes enhances package security but might lead to longer travel distances and higher energy usage. Similarly, flying at lower altitudes conserves energy but increases the risk of obstacles.

Example 3 —Multi-Objective Learning

Multi-objective optimization plays a crucial role in machine learning, particularly in multi-objective learning (MOL), where a model is trained to perform well on multiple objectives simultaneously. At a higher level, the learning system seeks to develop a model that generalizes well across all objectives. A common scenario in multi-objective learning could be:

- *Maximize Accuracy:* The model is tuned to provide the most accurate predictions possible.
- *Maximize Fairness Score:* The model aims to mitigate biases that could lead to discriminatory decisions against certain groups (Pessach and Shmueli 2022).
- *Minimize Power Consumption:* The model is optimized to minimize energy usage, which is critical for applications such as embedded systems and large-scale data centers (Ghosh et al. 2013).
- *Minimize Latency:* The model seeks to reduce inference time or transmission delay, which is essential for real-time applications such as autonomous driving and cloud computing (Sagduyu et al. 2024).

These objectives are often interdependent. For instance, improving accuracy may increase latency due to more complex computations, while enhancing

fairness could reduce accuracy if certain constraints limit model flexibility. Similarly, minimizing power consumption may lead to slower inference speeds, impacting real-time performance. Addressing these trade-offs requires careful multi-objective optimization to achieve a well-balanced model that meets application-specific requirements.

Example 4 - Federated Learning

Multi-objective optimization also plays a crucial role in federated learning, for example in the healthcare domain. The goal is to train a model across multiple hospitals while addressing trade-offs between prediction risk, privacy, and interpretability. The objectives in this example are:

- *Minimize Prediction Risk:* The model should generalize well across heterogeneous hospital datasets, each exhibiting different biases, missing data patterns, and patient demographics.
- *Maximize Privacy Guarantees:* Due to regulations, patient data cannot be shared directly. Federated learning enables decentralized training, but techniques such as differential privacy degrade model accuracy (De Cristofaro 2020).
- *Maximize Model Interpretability:* In high-stakes applications like healthcare, clinicians require interpretable models to understand the model's decisions (Gilpin et al. 2018).

These objectives are inherently conflicting. For instance, a complex deep learning model may minimize risk but at the cost of low interpretability. Strong differential privacy guarantees improve security but increase prediction risk.

All of these examples have shown that multi-objective optimization finds a wide range of applications and that it requires balancing competing objectives. To address this trade-off, we are interested in finding Pareto-optimal solutions, where improving one objective may come at the cost of another. The set of all Pareto-optimal solutions is the Pareto set and it usually contains more than a single solution. The image of the Pareto set under the objective functions is known as the *Pareto front*.

1.2 Open Problems

1.2.1 Optimization toward the Pareto set

In finding an optimal model that generalizes across multiple objectives, a decision-maker typically seeks a Pareto-optimal solution. One approach is to approximate the entire Pareto set, allowing them to select a model that best fits their specific needs (Ma et al. 2020). However, this method is often

impractical and computationally expensive as the number of objectives grow (Opris et al. 2024). Additionally, approximating the entire Pareto set does not directly solve the preference a user may have. Therefore, we explore alternative approaches to achieve Pareto-optimality more efficiently.

In the often cited literature surrounding *Optimization toward the Pareto set*, an initial point is refined into a Pareto-optimal solution. The simplest approach is *Linear Scalarization*, which minimizes the weighted sum of the objectives. However, linear scalarization can only find Pareto-optimal solutions that lie on the convex envelop of the Pareto front (Boyd and Vandenberghe 2004), and hence does not give a complete profiling of the Pareto front when the objective functions are non-convex.

There are numerous examples of gradient-based algorithms, some of which select an arbitrary solution from the Pareto set, such as *Multiple Gradient Descent (MGD)* Désidéri (2012) and *Bilevel Pareto Descent Optimization (PDO)* (Mahdi Kamani et al. 2021). These methods usually require smooth and convex objective functions to ensure convergence guarantees, although they can still be applied to non-convex problems with weaker guarantees.

However, there is also a gradient-based algorithm that incorporates user preferences, yielding a Pareto-optimal solution that aligns with these preferences, provided that such a solution exists and the objective functions satisfy continuously differentiability (Mahapatra and Rajan 2020). The preference acts as a weighting measure, determining how much importance is assigned to each objective.

To date, no algorithm has been proposed that guarantees convergence to a Pareto-optimal solution while simultaneously optimizing an additional criterion function. A criterion function could encode the user's preferences, which do not necessarily need to depend on the objectives and, therefore, cannot easily be transformed into preferences as described above.

1.2.2 Optimization within the Pareto set

A relatively unexplored topic in research is *Optimization within the Pareto set*. Suppose a Pareto-optimal solution has already been obtained, but the user wishes to readjust the trade-offs between objectives. One approach is to restart the optimization process to obtain a new solution. However, this method could be computationally expensive, as it does not leverage the previously computed Pareto-optimal point. Moreover, convergence to the desired solution is not guaranteed.

A decision maker may have the alternative goal to optimize an additional criterion function within the Pareto set. Since no algorithm currently exists for optimization toward the Pareto set incorporating the criterion function,

there is also no established approach for restarting the optimization process in this context.

A naive approach to optimize within the Pareto set is to apply Riemannian manifold gradient descent directly on the Pareto set. However, this method incurs a high computational cost due to the need for eigen-decomposition of Hessian matrices at each iteration [Bonnabel (2013)].

Two recent approaches address this problem of optimizing an additional criterion function within the Pareto set: *Pareto Navigation Gradient Descent (PNGD)* (Ye and Liu 2022) and *Pareto Majorization-Minimization (PMM)* (Roy et al. 2023). However, Roy et al. (2023) demonstrated the existence of a counterexample where PNGD fails to converge. PMM, on the other hand, guarantees convergence under certain assumptions, including strong convexity for the objective functions, as well as twice differentiability and Lipschitz-continuous gradients.

1.2.3 Model Merging

The availability of large pre-trained base models has increased across many real-world applications (Thirunavukarasu et al. 2023, Cui et al. 2024). This has led to fine-tuning these models to adapt them for various specialized tasks such as computer vision (Zhai et al. 2023) or language generation (Roziere et al. 2023), and share the resulting fine-tuned models without sharing the underlying training data.

As the number of fine-tuned models grows, merging models with identical structures has gained attention as a method for integrating their strengths (Ilharco et al. 2022). This is particularly valuable in scenarios where training data remains confidential and cannot be shared, such as medical records in healthcare.

Existing model-merging methods primarily focus on improving the average task accuracy (Polikar 2012). However, conflicts between the objectives of different tasks can introduce trade-offs during model merging. Therefore, we seek for an optimal model on these objectives. MAP (Lu Li and Wang 2024) is a recently developed algorithm designed to address this challenge. MAP generates a set of Pareto optimal models, rather than a single model, allowing decision-makers to select a model that aligns best with their specific preferences in real-world applications.

However, MAP provides an approximation of the Pareto set, and no existing method optimizes directly within the Pareto set. This is where algorithms optimizing within the Pareto set become relevant. A decision maker can provide a criterion for which it seeks a Pareto-optimal model. We will setup an experimental pipeline to cooperate preferences into model merging by optimizing within the Pareto set. Such an approach is novel.

1.3 Research Question

Since only two algorithms have been proposed for optimization within the Pareto set, and the first fails to converge on a simple example, there is no absolute clear understanding of optimization within the Pareto set. To gain better insights, we focus on the well-established literature on optimization toward the Pareto set.

We compare these algorithms by implementing them in different example settings, including two convex objectives, two concave objectives, and three convex objectives that form a three-dimensional Pareto front. By analyzing these cases, we highlight similarities and structural components of these algorithms and compare them with optimization within the Pareto set. Furthermore, as Roy et al. (2023) theoretically proves that PNGD fails to converge on an example with two quadratic objective functions, we will verify this experimentally.

PMM, the most recent algorithm for optimization within the Pareto set, has not yet exhibited convergence issues. To further investigate its properties, we conduct an experiment analyzing the runtime as a function of the feature size. This type of runtime comparison has not been explored previously, and we aim to identify the computational bottlenecks within PMM.

Finally, we examine the MAP algorithm, which aims to optimally merge models to construct Pareto-optimal solutions. Currently, no approach incorporates user preferences while optimizing within the Pareto set. To address this gap, we propose an extension of the MAP algorithm by integrating PMM, enabling the generation of preference-based Pareto-optimal solutions. This type of construction has not yet been explored.

Chapter 2

Preliminaries of Multi-Objective Optimization

As a convention, we use the notation $[m] = \{1, \ldots, m\}$ and denote the i-th standard unit vector by e_i.

2.1 Definitions & Preliminaries

In multi-objective optimization (MOO), the goal is to minimize a set of m objective functions:

$$\min_{x \in R^n} (f_1(x), f_2(x), \ldots, f_m(x)) \tag{2.1}$$

where each objective function $f_i : \mathbb{R}^n \to \mathbb{R}$.

For convenience, we define $F \equiv (f_1, \ldots, f_m)$. Since minimization over a vector is not well-defined, we instead seek solutions that cannot be dominated by any other solution, known as Pareto-optimal solutions..

Definition 2.1 (*Pareto Dominance*) *A point $x \in \mathbb{R}^n$ dominates another point $y \in \mathbb{R}^n$ if and only if*

$$\begin{aligned} &f_i(x) \leq f_i(y), \quad \forall i \in [m], \\ &\text{and } f_j(x) < f_j(y) \text{ for some } j \in [m]. \end{aligned} \tag{2.2}$$

A solution that is not dominated by any other feasible solution is considered optimal in a Pareto sense.

Definition 2.2 (*Global Pareto Optimality*) *A point $x \in \mathbb{R}^n$ is called globally Pareto optimal if there is no point $y \in \mathbb{R}^n$ such that y dominates x. The set of all globally Pareto-optimal points with respect to the function $F : \mathbb{R}^n \to \mathbb{R}^m$ is called the Pareto set of F, denoted as Pareto(F).*

For simplicity, we will omit the term *global* when referring to Pareto optimality in the following sections.

Definition 2.3 (*Local Pareto Optimality*) *A point $x \in \mathbb{R}^n$ is called locally Pareto optimal if there exists a neighborhood $U \subseteq \mathbb{R}^n$ of x such that there is no point $y \in U$ that dominates x.*

Proposition 2.4 (*Censor (1977)*) *If all objective functions are convex, any locally Pareto-optimal point is also globally Pareto-optimal.*

Throughout the following sections, we will use the objective functions introduced in Example 2.5 to illustrate certain multi-objective optimization algorithms. This quadratic objective function example comes from Roy et al. (2023) and is expected to cause PNGD to fail to converge, whereas PMM should succeed. We will verify this in Section X.

Example 2.5 *For $x \in \mathbb{R}^2$, we consider the two objectives f_1, f_2:*

$$\begin{aligned} f_1(x) &= \frac{1}{2}\|A(x+e_1)\|_2^2 = \frac{1}{2}(x+e_1)^T H(x+e_1), \\ f_2(x) &= \frac{1}{2}\|A(x-e_1)\|_2^2 = \frac{1}{2}(x-e_1)^T H(x-e_1) \end{aligned} \tag{2.3}$$

with the matrix

$$H = A^T A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}.$$

Definition 2.6 (*Pareto Front*) *The Pareto front of a MOO problem is the image of all Pareto-optimal solutions under the objective function mapping and defined as*

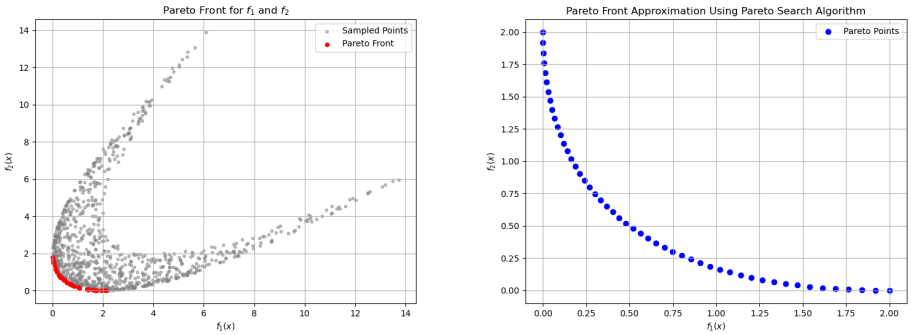
$$\mathcal{P}_F = \{F(x) \mid x \text{ is Pareto-optimal}\}. \tag{2.4}$$

The Pareto front of the objectives in Example 2.5 is visualized in Figure 2.1.

To form convex combinations of objective functions and their gradients, we require weights from the *probability simplex*. The probability simplex provides a natural way to express trade-offs in multi-objective optimization, as it ensures non-negative weights that sum to one.

Definition 2.7 (*Probability Simplex*) *The k-simplex, denoted as Δ^k, is the convex hull of $k+1$ standard unit vectors in $\mathbb{R}^{k+1}$, that is,*

$$\Delta^k = \left\{ \lambda \in \mathbb{R}^{k+1} \mid \sum_{i=1}^{k+1} \lambda_i = 1, \quad \lambda_i \geq 0 \text{ for all } i \right\}. \tag{2.5}$$



(a) Pareto front marked in red **(b)** Close-up image of the Pareto front

Figure 2.1: Pareto front of example 2.5

To find Pareto-optimal points, there is a similar notion to the single-objective case, where a local optimum implies that the gradient of the objective is zero. In the multi-objective setting, a locally Pareto-optimal point satisfies the condition of *Pareto stationarity*.

Definition 2.8 (*Pareto Stationarity*) *Assuming each f_i is continuously differentiable, we call a point $x \in \mathbb{R}^n$ Pareto stationary if zero is a convex combination of the gradients, that is,*

$$\sum_{i=1}^{m} \beta_i \nabla f_i(x) = 0 \quad \textit{for some } \beta \in \Delta^{m-1}. \tag{2.6}$$

Pareto stationarity is a widely used criterion in multi-objective optimization algorithms for identifying Pareto-optimal points. It ensures that no descent direction can simultaneously improve all objectives, making it a necessary condition for local optimality.

Proposition 2.9 (*Maruşciac (1982)*) *Assuming each f_i is continuously differentiable, all Pareto optimal points are Pareto stationary.*

Proposition 2.10 (*Maruşciac (1982)*) *Assuming each f_i is continuously differentiable, all Pareto stationary points are locally Pareto-optimal.*

To ensure that Pareto stationarity is a sufficient condition for Pareto optimality, additional assumptions are required.

Proposition 2.11 (*Fliege et al. (2009)*) *Assuming each $f_i(x)$ is twice-differentiable and strictly convex, a point x is Pareto optimal if and only if x is Pareto stationary.*

2.2 Optimization toward the Pareto set

2.2.1 Scalarization Technique

A simple approach is linear scalarization, where a convex combination of the objectives is constructed.

Definition 2.12 (*Linear Scalarization*) *Given the objective functions $F \equiv (f_1, ..., f_m)$ and convex weights $\beta \in \Delta^{m-1}$, we define the linear scalarization of F as*

$$f_\beta(x) := \sum_{i \in [m]} \beta_i f_i(x). \tag{2.7}$$

In the scalarization technique, a weight vector $\beta \in \Delta^{m-1}$ is given, and we aim to solve

$$\min_{x \in \mathbb{R}^n} f_\beta(x). \tag{2.8}$$

This technique only finds Pareto-optimal solutions that lie on the convex envelope of the Pareto front (see, e.g., (Boyd and Vandenberghe 2004)) and, hence, does not provide a complete representation of the Pareto front when the objective functions are non-convex.

Additionally, a major drawback of this approach is that the choice of parameters, such as the weight vector β, is not known in advance, placing the burden of selection on the decision-maker. Furthermore, changing β slightly does not necessarily result in a proportionally small change in the location of the new Pareto-optimal point, as the mapping from β to the Pareto front can be nonlinear (Shim et al. 2002).

2.2.2 Gradient-Based Algorithms

There are several gradient-based algorithms that optimize toward the Pareto set by starting with an initial point $x_0 \in \mathbb{R}^n$ and iteratively refining it toward the Pareto set, such that after T iterations, $x_T \in \text{Pareto}(F)$. These algorithms typically require the objective functions to be differentiable to compute gradients, and continuous differentiability is often assumed to ensure smooth gradient updates and stability in optimization. To guarantee convergence, it is helpful to have convex objectives, though this is not strictly required.

One such algorithm is the *Multiple Gradient Descent Algorithm (MGDA)* (Désidéri 2012), which is based on the steepest descent direction approach introduced by Fliege and Svaiter (2000). At each iteration, it takes a step along a direction that improves all objective functions in a balanced way. Specifically, it

can be shown (Göpfert and Nehse 1990, Luc 1988) that if a point $x \in \mathbb{R}^n$ is not locally Pareto-optimal, there exists a direction $v \in \mathbb{R}^n$ satisfying

$$-J_F(x)v \in \mathbb{R}^m_{>0}, \tag{2.9}$$

where J_F is the Jacobian of the function F, and $\mathbb{R}_{>0}$ denotes the set of strictly positive numbers. However, simple gradient-based algorithms provide very limited control, meaning that the decision-maker has no influence over the specific outcome in the Pareto set, which may be arbitrary.

One recent gradient-based algorithm which incorporates preferences as a weighting of the objectives is *Exact Pareto Optimal Search (EPOS)* (Mahapatra and Rajan 2020). For a given reference $l \in \mathbb{R}^m$, the solution of EPOS belongs to the set P_l, defined as

$$P_l = \{x \in Pareto(f_1, ..., f_m) : l_1 \cdot f_1(x) = ... = l_m \cdot f_m(x)\} \tag{2.10}$$

As long as the objectives are continuously differentiable and the locally Pareto-optimal point exists that aligns with the preferences, the algorithm converges. It is not guaranteed that the point is globally Pareto-optimal.

2.2.3 Bilevel Optimization Algorithms

There are algorithms that optimize toward the Pareto set by formulating the optimization problem as a *bilevel optimization problem*, such as *Bilevel Pareto Descent Optimization (PDO)*, where one optimization problem is nested within another, namely an inner and an outer level. Each level has its objective function, but the solution of the inner level is being used in the optimization of the outer level. PDO does not require convex objectives and guarantees convergence to a locally Pareto-optimal point. However, it does not provide any direct control over the specific outcome in the Pareto set.

2.2.4 Representative Subsample Algorithms

The Pareto set typically contains multiple solutions and previous algorithms provide little to no control over the final Pareto solution. Therefore, the decision-maker may prefer to postpone the decision-making process and first construct a representative subsample. This subsample approximates the Pareto set and reduces the number of solutions that must be examined before making a final decision (Lin et al. 2019, Kobayashi et al. 2019, Ma et al. 2020).

A recent algorithm which aims to approximate the Pareto set is the *Efficient Pareto Set Exploration Algorithm (EPSE)* (Ma et al. 2020). A key theoretical foundation of this algorithm relies on the following proposition from Hillermeier (2001), namely:

Proposition 2.13 (*Hillermeier (2001)*) *Assuming that F is smooth and x^* is Pareto optimal, consider any smooth curve $x(t) : (-\varepsilon, \varepsilon) \to \mathbb{R}^n$ in the Pareto set and passing x^* at $t = 0$, i.e. $x(0) = x^*$, then $\exists \alpha \in \mathbb{R}^m$ such that:*

$$\nabla^2 f_\beta(x^*) x'(0) = \sum_{i=1}^{m} \alpha_i \nabla f_i(x^*) \tag{2.11}$$

and β are the convex weights witnessing the stationary condition for the Pareto optimal point x^*.

As we explore the Pareto set, each point fulfills the Pareto stationarity condition. This means if β witnesses Pareto stationarity for point x, we know for (x, β):

$$\nabla f_\beta(x) = 0 \tag{2.12}$$

Furthermore, note that $x'(0)$ is a tangent direction on the Pareto set; therefore, the goal is to find the pair (x', α) and explore the Pareto set along this tangent direction.

2.3 Optimization within the Pareto set

Multi-objective optimization within the Pareto set focuses on optimizing an additional criterion function $f_0 : \mathbb{R}^n \to \mathbb{R}$ constraint to the Pareto set of $F \equiv (f_1, \ldots, f_m)$:

$$\min_{x \in \text{Pareto}(F)} f_0(x) \tag{2.13}$$

A solution to this problem is called a *preference optimal solution*.

Definition 2.14 (*Preference Optimality*) *Given the objectives $F \equiv (f_1, \ldots, f_m)$ and the criterion function f_0, the preference optimal solution x_{pref} is*

$$x_{pref} := \underset{x \in Pareto(F)}{\arg\min}\ f_0(x). \tag{2.14}$$

There are two algorithms that aim to address this problem. The first of the two algorithms is *Pareto Navigation Gradient Descent (PNGD)* from Ye and Liu (2022), which leverages the Pareto stationarity condition, a necessary but not sufficient condition for Pareto optimality in the case of non-convex objectives.

The algorithm relies on differentiable objective functions and does not assume convexity. Therefore, it focuses on local Pareto-optimal points, which correspond to the zero set of g in Equation 2.15. Note that Pareto stationarity implies local Pareto optimality (see Proposition 2.10).

$$g(x) := \min_{\beta \in \Delta^{m-1}} \|\nabla f_\beta(x)\|_2^2 \tag{2.15}$$

The PNGD algorithm uses the value of $g(x)$ as a measure of distance to the Pareto front, where a larger $g(x)$ indicates a greater distance from the Pareto set. This notion of using $g(x)$ as a distance measure to the Pareto front has also been employed in Désidéri (2012), Sener and Koltun (2018) and is the smallest-norm vector in the convex hull of all objective gradients.

When x is close to the Pareto set, namely if $g(x) > \theta_0$ for some threshold θ_0, PNGD fully prioritizes optimizing the criterion function f_0. However, if x is far from the Pareto set, the algorithm primarily focuses on finding a direction that moves towards it.

Regarding convergence, Roy et al. (2023) presents a counterexample with quadratic objectives where PNGD fails to converge. This counterexample is detailed in Example 2.15 and empirically tested in Section 5.1.2.

Example 2.15 *For $x \in \mathbb{R}^2$, let*

$$\begin{aligned} f_0(x) &= \frac{1}{2}\|x - e_2\|_2^2, \\ f_1(x) &= \frac{1}{2}\|A(x + e_1)\|_2^2, \\ f_2(x) &= \frac{1}{2}\|A(x - e_1)\|_2^2 \end{aligned} \tag{2.16}$$

with the matrix

$$H = A^T A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}.$$

The second, and more recent, algorithm is *Pareto Majorization-Minimization (PMM)* from Roy et al. (2023), which imposes strong assumptions on the objective functions. Specifically, all objectives must be strongly convex, twice differentiable, and have Lipschitz-continuous gradients and Hessians. Additionally, the criterion function f_0 must also have Lipschitz-continuous gradients. These assumptions are necessary to guarantee convergence.

The algorithm is based on the idea of the *Pareto Manifold* $\mathcal{P}(F) \subset \mathbb{R}^n \times \Delta^{m-1}$, defined as the zero set of

$$\mathcal{P}(F) = \{(x, \beta) : \nabla f_\beta(x) = 0\} \tag{2.17}$$

The Pareto manifold contains all points (x, β) such that x is Pareto stationary (see Definition 2.8) and β serves as its witness. Since all objectives are

strongly convex, Pareto stationarity is a sufficient condition for Pareto optimality (Proposition 2.11).

All Pareto-optimal points can be obtained by projecting the Pareto manifold onto its first component. Note that $\mathcal{P}(F)$ is the zero set of the mapping:

$$(x, \beta) \mapsto \nabla f_\beta(x). \tag{2.18}$$

Note that f_β is strongly convex, as it is a convex combination of strongly convex objective functions. Since f_β is strongly convex,

$$\beta \mapsto x^*(\beta) = \arg\min_{x \in \mathbb{R}^n} f_\beta(x) \tag{2.19}$$

is well-defined and yields a map $x^* : \Delta^{m-1} \to \text{Pareto}(F)$ by the implicit function theorem. The objectives are smooth and therefore, at the unique optimum $x^*(\beta)$, we have $(\nabla f_\beta \circ x^*)(\beta) = 0$. Hence, the stationary condition is true and $x^*(\beta)$ is Pareto optimal.

It is hence easily verified that 2.13 is equivalent to solving

$$\min_{\beta \in \Delta^{m-1}} (f_0 \circ x^*)(\beta) \tag{2.20}$$

and then using $x_{\text{pref}} = x^*(\beta)$.

2.4 Model Merging

Model Merging aims to find a way to combine multiple fine-tuned models with the same architecture, namely with the weights $\{\theta^i_{ft}\}_{i \in [N]}$, in such a way that the merged model parameterized by

$$\theta_m = \sum_{i=1}^{N} k_i \theta^i_{ft} \tag{2.21}$$

perform reasonable well on N tasks where $k_i \in \mathbb{R}$ is the weight on model i (Wortsman et al. 2022).

Ilharco et al. (2022) has introduced *task arithmetic*, an effective way to perform model merging. For the family of fine-tuned model weights $\{\theta^i_{ft}\}_{i \in [N]}$, we denote θ_{pre} as base model weights. The task vector for task i is defined as

$$v_i = \theta^i_{ft} - \theta_{\text{pre}}, \tag{2.22}$$

which is the element-wise difference between the pre-trained weights and the fine-tuned weights for task i. To perform model merging with task vectors, we can compute

$$\theta_m = \theta_{\text{pre}} + \sum_{i=1}^{N} c_i v_i, \tag{2.23}$$

where $c_i \in \mathbb{R}$ are scaling factors that have shown to be essential to the performance of the merged model (Yadav et al. 2023, Yang et al. 2023).

Given the task vectors $\{v_i\}_{i=1}^N$ and the base model weights $\theta_{\text{pre}} \in \mathbb{R}^d$, we write the weights of the merged-model θ_m as a function of the coefficients $c \in \mathbb{R}^N$, that is,

$$\theta_m(c) = \theta_{\text{pre}} + Vc, \text{ where } V = \text{concat}(v_1, ..., v_N) \in \mathbb{R}^{d \times N} \tag{2.24}$$

MAP (Lu Li and Wang 2024) assumes N tasks and for each task a separate objective function, that we aim to minimize. MAP computes a quadratic approximation of each of the objective functions by using the second-order Taylor expansion, this yields for task $n \in [N]$:

$$f_n(\theta_m(c)) \approx f_n(\theta_{\text{pre}}) + \nabla f_n(\theta_{\text{pre}})^\top Vc + \frac{1}{2}(Vc)^\top \nabla^2 f_n(\theta_{\text{pre}}) Vc \tag{2.25}$$

Our goal is to find optimal coefficients c such that $\theta_m(c)$ is Pareto-optimal. The task vectors V, the base-model weights θ_{pre} and $(f_n(\theta_{\text{pre}}), \nabla f_n(\theta_{\text{pre}}), \nabla^2 f_n(\theta_{\text{pre}}))$ are constant terms in this setting. This simplifies the expression to

$$\hat{f}_n(c; A_n, b_n, e_n) = e_n + b_n^\top c + \frac{1}{2} c^\top A_n c \tag{2.26}$$

with $e_n = f_n(\theta_{\text{pre}}) \in \mathbb{R}$, $b_n = V^\top \nabla f_n(\theta_{\text{pre}}) \in \mathbb{R}^N$ and $A_n = V^\top \nabla^2 f_n(\theta_{\text{pre}}) V \in \mathbb{R}^{N \times N}$ with $A_n^\top = A_n$. MAP evaluates the objective function f_n for different coefficients c and determines e_n, b_n, A_n such that the approximation $\hat{f}_n$ provides the best possible fit in a least-squares sense. With this well-defined approximation, it becomes possible to generate models that are Pareto-optimal.

2.5 Open Problems

There are numerous algorithms designed to optimize toward the Pareto set or approximate it. However, only two algorithms explicitly focus on optimizing an additional criterion function f_0 within the Pareto set.

We primarily focus on the most recent algorithms within specific subcategories of optimization toward the Pareto set. These categories include:

- *Representative Subsampling: Efficient Pareto Set Exploration (EPSE)* (Ma et al. 2020)
- *Gradient-Based Optimization with random Solution: Multiple Gradient Descent Algorithm (MGDA)* (Désidéri 2012)
- *Gradient-Based Optimization with Preference Vector: Exact Pareto Optimal Search (EPOS)* (Mahapatra and Rajan 2020)

- *Bilevel Optimization: Pareto Descent Optimization (PDO)* (Mahdi Kamani et al. 2021)

As an introduction to gradient-based techniques, we will also incorporate ideas developed in the steepest descent algorithm of Fliege and Svaiter (2000). A more recent extension of this basic descent approach is MGDA.

For optimization within the Pareto set, the only two existing algorithms are:

- *Pareto Navigation Gradient Descent (PNGD)* (Ye and Liu 2022)
- *Pareto Majorization-Minimization (PMM)* (Roy et al. 2023)

We will test these algorithms on different examples, including two convex objectives, two concave objectives, and three convex objectives, and compare the results. Additionally, we will replicate the experiment from Roy et al. (2023) on the divergence of PNGD, where the objectives are defined in Equation 2.16. This experiment was previously only proven theoretically in the original paper.

As PMM is the most recent algorithm for multi-objective optimization within the Pareto set, we will analyze its runtime behavior as a function of the feature size in an experimental setting. Such an analysis has not been conducted before.

Furthermore, we will incorporate PMM into model merging, building upon the groundwork established by MAP. MAP approximates the metrics using quadratic functions. We conduct a similar experiment as outlined in MAP but optimize directly within the Pareto set. The experiment involves language models, specifically a fine-tuned French model θ_{ft}^{F} and a fine-tuned Arabic model θ_{ft}^{A}. Our objective is to merge these models to achieve a Pareto-optimal balance between French and Arabic performance while also optimizing for Japanese, which represents the decision-maker's preference. Assuming the base model of both the fine-tuned models is θ_{pre}. In a first step, we extract the task vectors:

$$\begin{aligned} v_F &= \theta_{ft}^{F} - \theta_{\text{pre}} \\ v_A &= \theta_{ft}^{A} - \theta_{\text{pre}} \end{aligned} \tag{2.27}$$

The objectives that describe the performance on the French, Arabic and Japanese language are approximated as described in Section 2.4. We aim to minimize these objectives, which are formulated as follows:

$$\begin{aligned} f_0(c) &= e_{\text{Japanese}} + b_{\text{Japanese}}^{\top} c + c^{\top} A_{\text{Japanese}} c \\ f_1(c) &= e_{\text{French}} + b_{\text{French}}^{\top} c + c^{\top} A_{\text{French}} c \\ f_2(c) &= e_{\text{Arabic}} + b_{\text{Arabic}}^{\top} c + c^{\top} A_{\text{Arabic}} c \end{aligned} \tag{2.28}$$

We aim to find optimal coefficients c, such that

$$\min_{c \in \text{Pareto}(f_1, f_2)} f_0(c) \tag{2.29}$$

This experiment, which applies optimization within the Pareto set, is novel and will be further detailed in Section 5.1.4.

Chapter 3

Related Work

3.1 Optimization toward the Pareto set

3.1.1 Representative Subsampling

There are several approaches to optimize toward the Pareto set. As discussed in Section 2.2.4, one approach to finding Pareto-optimal points that satisfy a given preference is representative subsampling. After obtaining a good approximation of the Pareto set, the decision-maker can select the point that best aligns with their preferences.

However, as the number of objectives and dimensions increases, the Pareto set can grow exponentially, making it impractical to maintain a sufficiently representative subsample of a manageable size. Additionally, the geometry of the Pareto set can be highly complex, posing challenges for effective sampling (Kulkarni et al. 2023). Even in the case of quadratic objectives in two dimensions, singularities can arise in the Pareto set, further complicating the approximation process (Sheftel et al. 2013).

One of the most recent algorithms that approximates the Pareto set along a tangent direction on the Pareto front is *Efficient Pareto Set Exploration* (Ma et al. 2020). It requires twice differentiable objectives to find the pair (x', α) as discussed in Proposition 2.13. Even though it is possible to compute such a pair using classical $O(n^3)$ methods, such as the Gram-Schmidt process or QR decomposition, this remains challenging for two reasons. First, due to early termination, x' is rarely an exact Pareto-stationary solution, leading to inaccuracies. Second, in high-dimensional feature spaces, $O(n^3)$ methods become computationally prohibitive.

The paper proposes solving a slightly modified problem where they first compute α. However, even after finding α, it is still necessary to compute and invert the Hessian, which remains computationally expensive as discussed in Section 5.2.3.

Furthermore, the algorithm employs a black-box optimization method, denoted as *ParetoOptimize*(x), to project points onto the Pareto set. This point is usually already close to the Pareto set, as updates are performed along a tangent direction.

The subroutine *ParetoExpand*(x) computes the tangent direction at x. The parameter s denotes the step size used to update x along the tangent direction, while K represents the number of new points to be explored from a given point. A full illustration is in Algorithm 1.

Algorithm 1: Efficient Pareto Set Exploration (EPSE)

Input: Random initial point $x_0 \in \mathbb{R}^n$
Output: N Pareto-optimal points
$x_0^* \leftarrow ParetoOptimize(x_0)$
Initialize a queue $q \leftarrow [x_0^*]$
Initialize an empty list to store the output: $output \leftarrow \emptyset$
repeat
  Pop a Pareto-optimal point x^* from q
  for $i = 1$ *to* K **do**
    $v_i \leftarrow ParetoExpand(x^*)$
    $v_i / = \|v_i\|_2$
    $x_i \leftarrow x^* + sv_i$
    $x_i^* \leftarrow ParetoOptimize(x_i)$
    if *No points in output dominates* x_i^* **then**
      Append x_i^* to q
      Append $(x_i^*, F(x_i^*), \nabla F(x_i^*), x^*)$ to $output$
    end
  end
until *The size of output reaches* N

In other words, at each iteration, the algorithm selects a Pareto-optimal point from the queue, determines its tangent direction, and generates K new points along this direction. These points are then projected back onto the Pareto set, added to the queue, and stored as optimal solutions.

One downside of this algorithm is that it relies on a black-box optimization method, *ParetoOptimize*(x), which the user must choose. While the point is typically close to the Pareto set, there are cases where this is not true, such as when the initial point is far from the set.

3.1.2 Gradient-Based Algorithms

Gradient-based algorithms update in the direction of descent. A recent method that follows the descent direction approach is the *Multiple Gradient*

Descent Algorithm (MGDA) (Désidéri 2012). This algorithm requires differentiable objectives and iteratively refines a point toward the Pareto set.

As long as the point is not yet Pareto-stationary, there is a minimum-norm vector in the convex hull of the gradient vectors that is nonzero, providing a descent direction for all objectives simultaneously. The minimum-norm direction ensures a more stable update compared to earlier gradient-based algorithms that did not consider minimum-norm solutions, such as Fliege and Svaiter (2000).

The algorithm does not directly compute the minimum-norm vector within the convex hull of the gradients. Instead, it approximates the solution by solving the linear system:

$$G\beta = \mathbf{1}, \quad \sum_{i=1}^{m} \beta_i = 1, \quad \beta_i > 0 \tag{3.1}$$

where G is the Gram matrix containing the pairwise inner products of the objective gradients, and β represents the weight coefficients in the convex combination of gradients. However, solving this system does not guarantee minimization of the squared norm $\| \sum_{i=1}^{m} \beta_i \nabla f_i(x) \|_2^2$, but rather provides a heuristic approximation which computationally easier.

Moreover, MGDA is guaranteed to converge to a Pareto-stationary point which implies that the point is also locally Pareto-optimal. If the objectives are convex, the algorithm ensures convergence to a Pareto-optimal solution. Algorithm 2 illustrates the complete approach.

Algorithm 2: Multiple Gradient Descent Algorithm

Input: Random initial point $x \in \mathbb{R}^n$
Output: Pareto-optimal point
for $i = 1$ *to* T **do**
  Compute the gradient vectors $\nabla f_i(x)$ and determine the Gram matrix $G = (\nabla f_i(x)^\top \nabla f_j(x))_{1 \leq i,j \leq m}$
  Solve Equation (3.1) for β and compute $\omega = \nabla f_\beta(x)$
  $x \leftarrow x - h\omega$ where h is an optimal step size.
end
return x

A gradient-based algorithm which incorporates user preferences $l \in \mathbb{R}^m$ as a weighting of the objectives is *Exact Pareto Optimal Search (EPOS)* (Mahapatra and Rajan 2020). The outcome of this algorithm belongs to the set P_l, defined in Equation 2.10, as long as the objectives are continuously differentiable and the preference optimal point exists.

The algorithm follows an iterative approach toward the Pareto set while also progressing toward solutions that satisfy the condition in Equation 2.10.

To achieve this, the algorithm constructs a balancing direction v as a weighted sum of the gradients of $(f_1, \ldots, f_m)$, where the weights $\{a_i\}_{i=1}^m$ are referred to as adjustments (see Equation 3.4). Additionally, the algorithm requires the weighted normalization $\hat{f}_j$ (see Equation 3.2).

These adjustments are determined by the relative deviation of the *non-uniformity* measure $\mu_l(F)$, which quantifies how uneven the losses are (see Equation 3.3). This ensures that the algorithm converges toward an exact preference-specific Pareto-optimal solution.

$$\text{weighted normalization:} \quad \hat{f}_j = \frac{l_j f_j}{\sum_{j'=1}^m l_{j'} f_{j'}}, \tag{3.2}$$

$$\text{non uniformity:} \quad \mu_l(F) = \sum_{j=1}^m \hat{f}_j \log\left(\frac{\hat{f}_j}{1/m}\right), \tag{3.3}$$

$$\text{adjustments:} \quad a_j = l_j \left(\log\left(\frac{\hat{f}_j}{1/m}\right) - \mu_l(F)\right). \tag{3.4}$$

The non-uniformity measure quantifies how close a given point is to satisfying the preferences in Equation 2.10, with smaller values indicating greater proximity. Thus, we seek a direction such that moving against it from x results in a new solution with lower non-uniformity than x. The balancing direction,

$$d_{\text{bal}} = \sum_{j=1}^m a_j \nabla f_j(x) \tag{3.5}$$

achieves this goal.

From Theorem 1 in Mahapatra and Rajan (2020), we know that moving against any direction that forms a positive angle with d_{bal} improves uniformity. Furthermore, based on results from the gradient-based algorithm in Désidéri (2012), we know that the descent direction within the convex hull of the gradients of the objectives moves toward the Pareto set.

Combining these two requirements, the authors of EPOS formulate a linear program (LP) to determine a direction of improvement that satisfies both conditions.

The EPOS algorithm requires continuously differentiable objectives to ensure smooth updates at each iteration. It guarantees convergence to a locally Pareto-optimal solution that aligns with the preferences in Equation 2.10, provided such a point exists. However, there is no guarantee that the solution is globally Pareto-optimal.

3.1.3 Bilevel Optimization

There is an algorithm that uses bilevel optimization to find Pareto-optimal points, one of them is Bilevel Pareto Descent Optimization (PDO) (Mahdi Kamani et al. 2021). This algorithm considers

$$\begin{aligned} & x^* \in \arg\min_{x \in \mathbb{R}^n} \Psi(F(x), \beta^*(x)) = f_{\beta^*}(x) \\ \text{s.t.} \quad & \beta^* \in \arg\min_{\beta \in \Delta^{m-1}} \Phi(F(x), \beta) = \| \sum_{i=1}^{m} \beta_i \nabla f_i(x) \|_2^2 \end{aligned} \tag{3.6}$$

which consists of two nested optimization problems, namely an inner and an outer level. The inner level utilizes the stationarity condition, while the outer level optimizes the linear scalarization for a given β.

PDO does not require convex objectives and guarantees convergence to a locally Pareto-optimal point. Algorithm 3 illustrates the complete approach.

Algorithm 3: Bilevel Pareto Descent Optimization

Input: Initial weights $\beta^{(0)} \in \mathbb{R}^m$, initial solution $x^{(0)} \in \mathbb{R}^d$, step sizes ρ, η
Output: Pareto-optimal point
for $t = 0, 1, \ldots, T-1$ **do**
  for $k = 0, 1, \ldots, K-1$ **do**
    Compute $\beta^{(k+1)} = \beta^{(k)} - \rho \nabla_\beta \Phi(F(x^{(t)}), \beta)\Big|_{\beta=\beta^{(k)}}$
    Project onto the simplex: $\beta^{(k+1)} = \Pi_{\Delta^{m-1}}(\beta^{(k+1)})$
  end
  Set $\beta^{(t)} = \beta^{(K)}$
  Update solution: $x^{(t+1)} = x^{(t)} - \eta \nabla \Psi(F(x), \beta^{(t)})\Big|_{x=x^{(t)}}$
end
return $w^{(T)}$

For a given $x^{(t)}$ the inner loop aims to optimize $\beta^{(t)}$, the outer loop optimizes $x^{(t)}$. *Lemma 1* in Mahdi Kamani et al. (2021) states that the new optimum β_{opt} of the inner level ensures that the gradient of $f_\beta(x)$ is either zero or a descent direction for all objectives $f_i(x)$. This implies that at each step, the gradient of the outer level is either zero, indicating that a Pareto-stationary point has been reached, or a descent direction for all objectives. In the latter case, this descent direction can be used to decrease all objectives by moving in the negative gradient direction.

3.2 Optimization within the Pareto set

3.2.1 Pareto Navigation Gradient Descent

There are two algorithms that aim to optimize within the Pareto set by optimizing an additional criterion function $f_0 : \mathbb{R}^n \to \mathbb{R}$. The earlier of the two, Pareto Navigation Gradient Descent (PNGD) (Ye and Liu 2022), was published before the other. PNGD requires only differentiable objectives and therefore focuses on finding local Pareto-optimal solutions by leveraging the Pareto stationarity condition.

However, as previously mentioned, it fails to converge on an example with two quadratic objectives (see Example 2.15) constructed by Roy et al. (2023).

The algorithm follows an incremental updating rule of the form

$$x_{t+1} \leftarrow x_t - \eta v_t \tag{3.7}$$

where v_t is an update direction. Whenever x is far away from the Pareto set, the algorithm gives Pareto improvement to x and brings the point back to the Pareto front. If there are several directions yielding Pareto improvement, it chooses the one which decreases the preference function the most. In case the point x is very close to the Pareto set, it fully optimizes the preference function. This can be achieved by

$$\begin{aligned} v_t := \underset{v \in \mathbb{R}^n}{\arg\min} \quad & \left\{ \frac{1}{2} \|\nabla f_0(x_t) - v\|_2^2 \right\} \\ \text{subject to} \quad & \nabla f_i(x_t)^T v \geq \psi_t, \quad \forall i \in [m] \end{aligned} \tag{3.8}$$

The objective aims to bring the update direction v_t to as close as possible to an improvement step for the preference function f_0. If $\psi_t > 0$, the constraints can ensure that the update direction also decreases the objective values $f_1, ..., f_m$. Whenever we set $\psi_t = -\infty$, the constraints become vacuous and $v_t = \nabla f_0(x_t)$. The authors of this paper utilize the closeness-property of g, as discussed in Equation 2.15. This leads to set ψ_t accordingly

$$\psi_t = \begin{cases} -\infty & \text{if } g(x_t) \leq e, \\ \alpha_t g(x_t) & \text{if } g(x_t) > e. \end{cases} \tag{3.9}$$

where e is a small tolerance parameter and α_t is a positive hyper-parameter.

Stated under *Theorem 1* in Ye and Liu (2022), Equation 3.8 can be translated into its dual problem, which is more practical to solve. We get an update direction v_t, that has form

$$v_t = \nabla f_0(x_t) + \nabla f_{\beta_t}(x_t) \tag{3.10}$$

where β_t is the solution of the following dual problem

$$\beta_t = \arg\max_{\beta \in \mathbb{R}^n_+} -\frac{1}{2}\|\nabla f_0(x_t) + \nabla f_\beta(x_t)\|_2^2 + \psi_t \sum_{i=1}^{m} \beta_i \tag{3.11}$$

Algorithm 4 provides a full description.

Algorithm 4: Pareto Navigation Gradient Descent (PNGD)

Initialize: x_0; decide the step size η, and the control function ψ in Equation 3.9 (including the threshold $e > 0$ and the descanding rate $\{\alpha_t\}$).

for *iteration t* **do**

$$x_{t+1} \leftarrow x_t - \eta v_t, \quad \text{where} \quad v_t = \nabla f_0(x_t) + \nabla f_{\beta_t}(x_t)$$

where $\beta_{i,t} = 0, \forall i \in [m]$ if $g(x_t) \leq e$, and β_t is the solution of 3.11 with $\psi(x_t) = \alpha_t g(x_t)$ when $g(x_t) > e$.

end

3.2.2 Pareto Majorization-Minimization

The most recent algorithm that addresses the problem of optimizing a criterion function within the Pareto set is Pareto Majorization-Minimization (PMM) (Roy et al. 2023). PMM requires strongly convex and twice differentiable objectives, and Lipschitz-continuous gradients and Hessians. Additionally, the preference function f_0 must also have Lipschitz-continuous gradients. If this condition is met, the authors guarantee that the algorithm converges, provided that the black-box optimizers generate solutions within a certain error tolerance range.

As discussed in Section 2.3, the algorithm aims to solve the equivalent problem of Equation 2.13, namely:

$$\min_{\beta \in \Delta^{m-1}} (f_0 \circ x^*)(\beta) \tag{3.12}$$

and then sets $x_{\text{pref}} = x^*(\beta)$.

It is a smooth optimization problem on the simplex and it can be solved with gradient methods. The gradient of $(f_0 \circ x^*)(\beta)$ can be computed by using the chain rule. There is an issue that arises if we take the gradient of x^* with respect to β, namely it does not have a closed form. It uses the implicitly defined function $\beta \mapsto x^*(\beta)$ from Equation 2.19.

$$\nabla x^*(\beta) = -\nabla^2 f_\beta(x^*(\beta))^{-1} \nabla F(x^*(\beta))^\top \tag{3.13}$$

Therefore, the algorithm makes an approximation of the term $x^*(\beta)$ by introducing $\hat{\nabla} x^*(x, \beta)$, that is,

$$\hat{\nabla} x^*(x, \beta) = -\nabla^2 f_\beta(x)^{-1} \nabla F(x)^\top \tag{3.14}$$

The approximation at (x, β) for some distant x should not be as equally valid as one computed at $(x^*(\beta), \beta)$. The validity of the approximation is captured by using it to construct a majorizing surrogate function $g : \Delta^{m-1} \to \mathbb{R}$, which is a function that upper bounds $f_0 \circ x^*$, namely

$$(f_0 \circ x^*)(\beta) \leq g(\beta) \quad \text{for all } \beta \in \Delta^{m-1} \tag{3.15}$$

The standard quadratic upper bound for Lipschitz smooth functions (Nesterov 2013) yields a family of majorizing surrogates for g. This surrogate function will bound how much improvement in f_0 is made by any iterative optimization scheme that takes a step in β.

Algorithm 5 provides a full description. In each iteration, it constructs a surrogate function g_k and aims to minimize g_k over β, where g_k serves as an upper bound on the objective function f_0. Subsequently, the algorithm minimizes the linear scalarization of the objectives with the updated β over x. This ensures that at each iteration, the new point x remains in the Pareto set.

Algorithm 5: Pareto majorization-minimization (PMM)

Input: objectives $F \equiv (f_1, ..., f_m)$, preference function f_0, and black-box optimizer $\widehat{\operatorname{argmin}}$

Initialize: $(\beta_0, x_0) \in \Delta^{m-1} \times \mathbb{R}^n$

for $k = 1, ..., K$ **do**

Compute a majorizing surrogate $g_k(\beta) \equiv g(\beta; x_k, \beta_k)$

Compute approximate minimizers

$$\beta_{k+1} \leftarrow \widehat{\arg\min_{\beta \in \Delta^{m-1}}} g_k(\beta) \text{ and } x_{k+1} \leftarrow \widehat{\arg\min_{x \in \mathbb{R}^n}} f_{\beta_{k+1}}(x)$$

end

Under the strong assumption that the objectives and the criterion function satisfy the required conditions, as discussed at the beginning of this section, and assuming that the black-box optimizers for minimizing g_k and $f_{\beta_{k+1}}$ in the algorithm operate with a sufficiently small error, the algorithm guarantees an improvement in the criterion function at each iteration while ensuring that the updated point remains in the Pareto set.

3.3 Model Merging

A use case of multi-objective optimization within the Pareto set could be Model Merging, especially for an extension of the MAP algorithm (Lu Li and Wang 2024). Given N fine-tuned models with weights $\{\theta^i_{ft}\}_{i=1}^N$, all with the same architecture trained on the base model with weights θ_{pre}. MAP aims to merge these models and generate a set of Pareto-optimal models with reference to the N objective functions. As we have seen in Section 2.4, MAP first computes a quadratic approximation of the objective functions $\hat{f}_n$ as stated in Equation 2.26. There are $\frac{(N+1)(N+2)}{2}$ values missing to completely determine e_n, b_n and A_n, as A_n is a symmetric matrix. For various coefficients $c \in \Omega \subseteq \mathbb{R}^N$, MAP aims to find the best approximation in least-square terms:

$$A_n^*, b_n^*, e_n^* = \underset{A_n, b_n, e_n}{\arg\min} \sum_{c \in \Omega} |f_n(\theta_m(c)) - \hat{f}_n(c; A_n, b_n, c_n)|^2 \tag{3.16}$$

After having found a good quadratic approximation of the objective functions, MAP applies MOOP algorithms to determine the Pareto set. A full description can be found in Algorithm 6.

Algorithm 6: MAP

Prepare models $\{\theta^i_{ft}\}_{i=1}^N$ and compute task vectors $\{v_i = \theta^i_{ft} - \theta_{\text{pre}}\}_{i=1}^N$
for $n \in [N]$ **do**
  Sample K vectors of $c \in \mathbb{R}^N$. Denote the set as Ω.
  for $c = (c_1, \ldots, c_N) \in \Omega$ **do**
    Compute $\theta_m(c) = \theta_{\text{pre}} + c_1 v_1 + \cdots + c_N v_N$
    Derive the evaluation of objective funtion $f_n(\theta_m(c))$.
  end
  Fit the quadratic approximation model $\hat{f}_n$ by computing A_n, b_n, e_n in (3.16).
end
Apply MOOP algorithm (e.g. NSGA-3 (Yuan et al. 2014)) to $\{\hat{f}_i\}_{i=1}^N$ and get the Pareto set.

Chapter 4

Comparison & Building Blocks

4.1 Descent Direction

Optimization toward the Pareto set and optimization within the Pareto set are closely related. Gradient-based techniques are employed both for optimizing toward the Pareto set (e.g., MGDA, EPOS) and for optimizing within the Pareto set (e.g., PNGD).

The goal is to find a descent direction $v \in \mathbb{R}^n$ for a given point $x \in \mathbb{R}^n$ that improves all objectives. It can be shown (Göpfert and Nehse 1990, Luc 1988) that if x is not locally Pareto-optimal, then the descent direction v satisfies

$$-J_F(x)v \in \mathbb{R}^m_{>0}. \tag{4.1}$$

This condition ensures that v simultaneously improves all objectives. This idea is often used as a building block, and various techniques exist for determining such a descent direction, some of which we discuss next.

Optimization Problem in Steepest Descent

One of the earliest gradient-based algorithms, proposed by Fliege and Svaiter (2000), formulates a constrained optimization problem to determine a descent direction v as in Equation 4.1. It aims to minimize a scalar α and squared norm of the vector v, ensuring that the descent rate along v does not exceed α for each objective. Suppose we are given a point $x \in \mathbb{R}^n$, then the update rule is given by:

$$\begin{aligned} x_{t+1} &\leftarrow x_t + \eta v_t, \\ (\alpha, v_t) &:= \underset{(\alpha,v)\in\mathbb{R}^{n+1}}{\arg\min} \quad \alpha + \frac{1}{2}\|v\|_2^2 \\ \text{subject to} \quad & (J_F(x)v)_i \leq \alpha, \end{aligned} \tag{4.2}$$

which defines a convex quadratic problem with linear inequality constraints, where η is the step size.

The paper demonstrates that if x is not locally Pareto-optimal, then $\alpha(x) < 0$, meaning that taking an update step along v with a sufficiently small η improves all objectives. This result follows directly from Equation 4.1. In the case where x is Pareto-optimal, we obtain $v(x) = \alpha(x) = 0$. This optimization problem was originally introduced as one of the earliest gradient-based algorithms for multi-objective optimization. However, it has since been adapted and incorporated into newer algorithms in modified forms.

Optimization Problem in PNGD

One algorithm that builds upon a modified version of this approach is PNGD (Ye and Liu 2022), which aims to optimize a criterion function f_0 within the Pareto set. If a point $x \in \mathbb{R}^n$ is not locally Pareto-optimal, Equation 4.1 implies the existence of a descent direction $v \in \mathbb{R}^n$ such that an update step along v improves all objectives, as stated in Equation 4.1.

PNGD seeks to find a v that not only ensures movement toward the Pareto set but also aligns as closely as possible with the gradient of f_0, thereby improving f_0 simultaneously. This leads to the following optimization problem:

$$\begin{aligned} x_{t+1} &\leftarrow x_t - \eta v_t, \\ v_t &:= \underset{v \in \mathbb{R}^n}{\arg\min} \quad \frac{1}{2}\|\nabla f_0(x_t) - v\|_2^2 \\ \text{subject to} \quad & \nabla f_i(x_t)^T v \geq \psi, \quad \forall i \in [m], \end{aligned} \tag{4.3}$$

where η is the step size. Setting $\psi > 0$ ensures that v improves all objectives. Roy et al. (2023) shows a counterexample where this technique does not converge to the *preference* optimal solution. However, it still converges to a locally Pareto-optimal solution. Compared to the steepest descent optimization problem in Equation 4.2, the constraints use $\geq$ instead of $\leq$. This difference arises because, in steepest descent, we seek a descent direction, whereas in PNGD, we search for an ascent direction that is subsequently subtracted, following the same underlying principle.

Optimization Problem in MGDA

MGDA (Désidéri 2012) is a gradient-based algorithm that, given a point $x \in \mathbb{R}^n$, aims to find a descent direction v toward the Pareto set as stated in Equation 4.1. Compared to the previous algorithms, it determines the unique smallest-norm direction within the convex hull of the objective gra-

dients, given by:

$$x_{t+1} \leftarrow x_t - \eta \sum_{i=1}^{m} \beta_i^* \nabla f_i(x_t),$$
$$\text{where} \quad \beta^* := \underset{\beta \in \Delta^{m-1}}{\arg\min} \left\| \sum_{i=1}^{m} \beta_i \nabla f_i(x_t) \right\|_2^2. \tag{4.4}$$

The smallest-norm direction provides a well-balanced descent direction for all objectives, ensuring that progress is distributed evenly across them (Désidéri 2012).

Optimization Problem in PDO

The smallest-norm direction within the convex hull of the objective gradients also appears in PDO (Mahdi Kamani et al. 2021), an algorithm which employs a bilevel optimization problem, formulated as:

$$x^* \in \underset{x \in \mathbb{R}^n}{\arg\min} f_{\beta^*}(x)$$
$$\text{subject to} \quad \beta^* \in \underset{\beta \in \Delta^{m-1}}{\arg\min} \left\| \sum_{i=1}^{m} \beta_i \nabla f_i(x) \right\|_2^2. \tag{4.5}$$

Intuitively, this approach differs from typical gradient-based methods which aim to find a descent direction as stated in Equation 4.1. The algorithm solves this bilevel optimization problem using a nested loop, where the inner loop optimizes β in Equation 4.6 for a given x, while the outer loop updates x toward the Pareto set with one gradient step (see full description in Algorithm 3).

The implementation of PDO follows the same approach as MGDA. Solving for optimal β in Equation 4.6 is equivalent to finding the weights β for the smallest-norm direction v within the convex hull of the objective gradients. Both algorithms update x by moving along this direction and therefore follow a descent direction.

Thus, although the problem of finding a locally Pareto-optimal solution is formulated differently in each case, the algorithms operate in the same manner. Furthermore, both algorithms guarantee convergence to a Pareto-optimal solution, albeit through different proof techniques.

MGDA does not directly optimize the problem stated in Equation 4.6 to find the smallest-norm direction within the convex hull of the objective gradients. Instead, it first finds the Gram matrix and then approximates this solution to gain better runtime (see Désidéri (2012)). Therefore, the solution is not exactly the same as the one from PDO if both algorithms are run on the same example, tested in Section 5.1.1.

4.2 Pareto Stationarity

This is how we transition from finding a descent direction to achieving Pareto stationarity. Namely, if zero lies in the convex hull of the objective gradients, the point is called Pareto stationary. Proposition 2.9 states that Pareto stationarity is a necessary condition for Pareto optimality. Many algorithms aim to find such Pareto stationary points and optimize for the best β in Equation 4.6. This tool is often used as a fundamental building block in various algorithms, as we ultimately seek to find the origin within the convex hull.

$$\min_{\beta \in \Delta^{m-1}} \|f_\beta(x)\|_2^2. \tag{4.6}$$

As discussed in Section 4.1, Pareto stationarity is closely related to finding optimal weights for the smallest-norm direction within the convex hull of the objective gradients.

4.3 Optimization within the Pareto Set

Until now, only two algorithms have been designed to optimize within the Pareto set. The first, PNGD, does not converge on a quadratic example, as experimentally shown in Section 5.1.2. It utilizes a descent direction balanced with the gradient of the preference function.

A fundamentally different approach is taken by PMM, the second algorithm designed for optimization within the Pareto set. It seeks a preference-optimal solution by solving the equivalent problem of Equation 2.13, namely:

$$\min_{\beta \in \Delta^{m-1}} (f_0 \circ x^*)(\beta), \tag{4.7}$$

and then setting $x_{\text{pref}} = x^*(\beta)$.

The algorithm approximates the gradient of this optimization problem, which involves the composition of two functions. Consequently, it cannot be directly connected to the strategies used in previous algorithms.

As shown in Equation 3.14, the algorithm relies on the inversion of a Hessian matrix. Later, in Section 5.1.3, we will experimentally analyze how this computation may dominate the runtime.

4.4 Outcomes

Optimizing for the best β in Equation 4.6 is a fundamental building block in many algorithms, whether they are gradient-based, bilevel optimization-based, or designed for optimization within the Pareto set. Many algorithms

rely on the stationarity condition as a necessary condition for Pareto optimality. A Pareto stationary point is also a locally Pareto optimal point (Proposition 2.10). Additionally, optimal β values yield the weights for a descent direction (Equation 4.1), namely the unique smallest-norm direction within the convex hull of the objective gradients. This can be used to iteratively improve the point or serve as a measure of the distance to the Pareto set.

Chapter 5

Experiments

5.1 Experimental Setup

We first illustrate different algorithms that optimize toward the Pareto set. A comparative study of the latest algorithms, each employing distinct techniques, has not been conducted before. Our goal is to assess whether these algorithms successfully converge across different example settings, including convex, concave, and three-dimensional Pareto fronts and compare the different convergence paths.

Next, we empirically demonstrate the non-convergence of PNGD on an example provided by Roy et al. (2023), which was only shown theoretically in the paper. We then analyze the runtime of PMM as a function of feature size, providing insights into its computational efficiency as this algorithm is the newest approach for optimization within the Pareto set with convergence guarantees. Finally, we apply PMM in the context of model merging for MAP, an approach that has not been explored before. The code is available at https://github.com/silvanbolt/MOO.

5.1.1 Optimization toward the Pareto Set

We evaluate EPSE, MGDA, PDO, and EPOS on three different examples. The first example setting is from Roy et al. (2023), originally used to demonstrate the divergence of the PNGD algorithm. The example is stated in Example 2.15.

The second example considers two concave objectives and examines the behavior of the four algorithms. This example setting is newly introduced. The third example consists of three quadratic objectives that create a Pareto front in three dimensions, which is also newly introduced.

A comparative study of this kind does not exist in the literature and will provide valuable insights into the behavior of these algorithms.

Convex Example in 2D

We test EPSE, MGDA, PDO and EPOS on example 2.15.

For EPSE, we used $K = 2$ and $K = 10$ exploration directions per point in the algorithm's queue and generated a total of $N = 20$ points in the Pareto set.

For MGDA and PDO, an initial starting point is required, which we set as $x_1 = (-1.4, 0.9)$ and $x_2 = (-0.6, 1.4)$. The step size was chosen as $\eta = 0.1$, with $T = 50$ iterations.

For EPOS, we again initialized at $x_1 = (-1.4, 0.9)$ and $x_2 = (-0.6, 1.4)$, but with $T = 500$ iterations and a step size of $\eta = 0.01$. For each point, we used the two preference vectors $l_1 = (0.5, 0.5)$ and $l_2 = (0.1, 0.9)$.

The results are illustrated in Figure 5.2,5.3.

Concave Example in 2D

This time, we evaluate EPSE, MGDA, PDO, and EPOS on concave objective functions, defined for $x \in \mathbb{R}^2$:

$$\begin{aligned} f_1(x) &= 1 - \exp\left(-\|x - q\|^2\right), \\ f_2(x) &= 1 - \exp\left(-\|x + q\|^2\right), \qquad q = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right). \end{aligned} \tag{5.1}$$

For EPSE, we used $K = 2$ and $K = 10$ exploration directions per point in the algorithm's queue and generated a total of $N = 20$ points in the Pareto set.

For MGDA and PDO, an initial starting point is required, which we set as $x_1 = (-1.4, 0.9)$ and $x_2 = (-1.6, 0.2)$. The step size was chosen as $\eta = 0.1$, with $T = 200$ iterations.

For EPOS, we initialized at $x_1 = (-1.4, 0.9)$ and $x_2 = (-1.6, 0.2)$ with a step size of $\eta = 0.01$. For each point, we used the two preference vectors $l_1 = (0.5, 0.5)$ and $l_2 = (0.1, 0.9)$. We set the number of iterations to $T = 1000$, except for x_2 with l_1, where we used $T = 2000$, as it did not fully converge with only $T = 1000$.

The results are illustrated in Figure 5.4,5.5.

Example in 3D

We evaluate EPSE, MGDA, PDO, and EPOS on three quadratic objective functions that create a Pareto front in three dimensions, defined for $x \in \mathbb{R}^3$

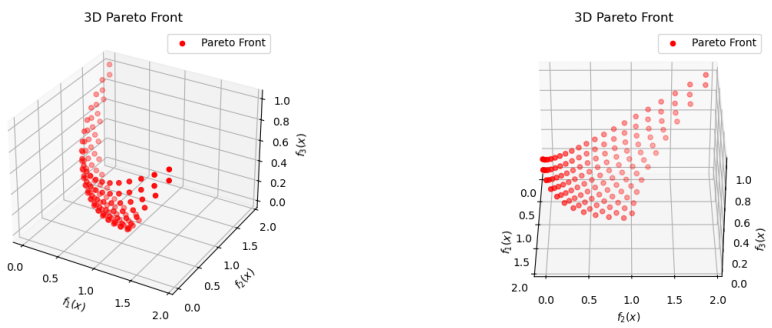
as:

$$f_1(x) = \frac{1}{2}\|(x + e_1)\|^2, \tag{5.2}$$

$$f_2(x) = \frac{1}{2}\|(x - e_1)\|^2, \tag{5.3}$$

$$f_3(x) = \frac{1}{2}\|(x - e_2)\|^2. \tag{5.4}$$

The Pareto front is visualized in Figure 5.1.



(a) View Angle 1 **(b)** View Angle 2

Figure 5.1: Pareto Front of Example 5.2

For EPSE, we used $K = 2$ and $K = 50$ exploration directions per point in the algorithm's queue and generated a total of $N = 200$ points in the Pareto set.

For MGDA and PDO, we initialized at $x_1 = (-0.8, 0.6, 0.4)$ and $x_2 = (0.2, 0.9, 0.5)$, with $T = 500$ iterations and a step size of $\eta = 0.1$.

For EPOS, we initialized at $x_1 = (-0.8, 0.6, 0.4)$ and $x_2 = (0.2, 0.9, 0.5)$ with a step size of $\eta = 0.01$ and $T = 1000$ iterations. For each point, we used the two preference vectors $l_1 = (0.33, 0.33, 0.33)$ and $l_2 = (0.1, 0.1, 0.8)$.

The results are illustrated in Figure 5.6, 5.7.

5.1.2 Example of Non-Convergence for PNGD

We implement the counterexample stated in Roy et al. (2023) to test the non-divergence of PNGD. While the original paper provides a theoretical guarantee, no experimental verification has been conducted to date. The example is given in Equation 2.16.

The objective f_0 is the criterion function. We first examine the Pareto front of f_1 and f_2. Lemma 6 in Roy et al. (2023) states that

$$\text{Pareto}(f_1, f_2) = \text{conv}(e_1, -e_1). \tag{5.5}$$

It is straightforward to see that minimizing the criterion function within the Pareto set requires selecting points that are as close as possible to e_2 in terms of the ℓ_2-norm. This is achieved by projecting e_2 onto $\text{Pareto}(f_1, f_2)$, which results in the origin as the preference-optimal solution:

$$x_{\text{pref}} = (0, 0). \tag{5.6}$$

This results in the following image point on the Pareto front:

$$F_{\text{pref}} = (f_1(x_{\text{pref}}), f_2(x_{\text{pref}})) = (\frac{1}{2}, \frac{1}{2}). \tag{5.7}$$

PNGD does not converge to the preference-optimal point, whereas PMM does. A demonstration of this implementation is shown in Figure 5.8, where the left side shows PMM and the right side shows PNGD. Roy et al. (2023) proves that PNGD converges to $x = e_1$ instead of the preference-optimal point $x_{\text{pref}} = (0, 0)$.

5.1.3 Runtime of PMM

As PMM is the most recent algorithm and has not yet been counterproved in terms of convergence, it remains an open question whether its guarantees hold universally. PMM requires computing the inversion of a Hessian at each iteration, which we suspect may dominate runtime compared to PNGD.

We aim to analyze the runtime dependency on the feature size, an aspect that has not been explored before. For this, consider the example described in Equation 5.8, which generates a family of functions $\{f_i\}_{i=1}^{2m}$ for a predefined n, where $f_i : \mathbb{R}^n \to \mathbb{R}$:

$$f_0(x) = \frac{1}{2}\|x - e_1\|_2^2,$$

For all $1 \leq i \leq m$:

$$\begin{aligned} f_{2i-1}(x) &= \frac{1}{2}\|A(x - e_{i+1})\|_2^2, \\ f_{2i}(x) &= \frac{1}{2}\|A(x + e_{i+1})\|_2^2. \end{aligned} \tag{5.8}$$

where

$$A = \text{diag}(1, 2, 3, ..., n). \tag{5.9}$$

Proposition 5.1 *Given the family of objectives functions $\{f_i\}_{i=1}^{2m}$ from Equation 5.8, then the following holds for the Pareto set*

$$Pareto(f_1, \dots, f_{2m}) = conv(e_2, -e_2, \dots, e_{m+1}, -e_{m+1}). \tag{5.10}$$

Proof. Recall Lemma 6 from Roy et al. (2023) that given objectives $L \equiv (l_1, \dots, l_k)$ with $l_i : \mathbb{R}^n \to \mathbb{R}$ and

$$l_i(x) = \frac{1}{2}\|A(x - z_i)\|_2^2, \quad \text{with full Rank } A \in \mathbb{R}^{n \times n} \text{ and } z_i \in \mathbb{R}^n \tag{5.11}$$

then the Pareto set of L is the convex hull of all z_i, namely

$$\text{Pareto}(l_1, \dots, l_k) = \text{conv}(z_1, \dots, z_k) \tag{5.12}$$

In case of objectives $f_1, \dots, f_{2m}$, we have $z_i = (-1)^{i+1} e_{\lceil i/2 \rceil + 1}$ for $i \in [m]$ and therefore it is easy to verify that the proposition is valid.

By the simple construction of f_0, the optimum of $f_0(x)$ for $x \in$Pareto$(f_1, \dots, f_{2m})$ is achieved, when x has minimum ℓ_2-distance to e_1. Therefore, we project e_1 onto Pareto$(f_1, \dots, f_{2m})$=conv$(e_2, -e_2, \dots, e_{m+1}, -e_{m+1})$.
As conv$(e_2, -e_2, \dots, e_{m+1}, -e_{m+1}) \subseteq U$ where U is the subspace spanned by $e_2, \dots, e_{m+1}$ and e_1 is in the orthogonal complement of U, the projection maps to the origin.

$$x_{\text{pref}} = 0 \in \mathbb{R}^n \tag{5.13}$$

The experiment analyzes runtime as a function of the feature size n. To achieve this, we first fix the number of objectives k and then construct $F_k = (f_1, \dots, f_k)$ for different feature sizes n.

We measure the time required for the initial point x_0 to converge to the preference-optimal point $0 \in \mathbb{R}^n$ within an error tolerance of at most ε. This leads to the stopping criterion: convergence is achieved as soon as x_t satisfies $\|x_t\| \leq \varepsilon$. An illustration of the runtime can be seen in Figure 5.9.

5.1.4 Optimization within the Pareto set for Model Merging

MAP (Lu Li and Wang 2024) is an algorithm which merges N models and computes optimal coefficients $c \in \mathbb{R}^N$, such that the merged model with weights θ_m

$$\theta_m(c) = \theta_{\text{pre}} + c_1 v_1 + \dots + c_N v_N \tag{5.14}$$

is Pareto-optimal. The algorithm provides an approximation of the Pareto set. However, until now, no adaptation of this algorithm has been designed to optimize within the Pareto set. One may seek to optimize an additional

criterion function f_0 and determine the coefficients such that the resulting model remains Pareto-optimal while aligning with the minimum of the preference function.

We implement an in-Pareto optimization algorithm that builds upon the core idea of MAP. Similar to MAP, we first compute quadratic approximations of the objective functions. However, instead of applying traditional multi-objective optimization (MOOP) algorithms, we optimize within the Pareto set with respect to an additional criterion function.

We conduct a similar experiment to MAP's language construction task, utilizing the same language base models, namely the Llama3 model. Instead of objective functions, we use metrics to evaluate model performance: f_1 represents the French perplexity measure, while f_2 represents the Arabic perplexity measure.

Perplexity is a measure of how well a language model predicts a given text; a lower perplexity value indicates better predictive performance, as the model assigns higher probabilities to the correct sequence of words. Additionally, we require fine-tuned versions of Llama3 for French and Arabic, which we aim to merge based on the metrics f_1 and f_2.

While MAP suggests generating the Pareto set of optimal solutions by merging the French and Arabic models, we extend this approach by introducing an additional preference metric f_0: the perplexity measure for Japanese. Our goal is to merge the French and Arabic models to construct a Pareto-optimal model that optimally performs on the Japanese perplexity measure. We first create a quadratic approximation of f_0, f_1, f_2. Using PMM, we then search for a preference Pareto-optimal solution.

All models can be found on Hugging Face. The IDs of the models are:

- *Base Model*: `meta-llama/Llama-3.1-8B`
- *French*: `jpacifico/French-Alpaca-Llama3-8B-Instruct-v1.0`
- *Arabic*: `MohamedRashad/Arabic-Orpo-Llama-3-8B-Instruct`

For each perplexity measure including French, Arabic and Japanese, we required a corresponding language dataset to compute this metric. The datasets are publicly available and can be found here:

- *French*: `huggingface.co/datasets/asi/wikitext_fr/blob/main/wikitext`
- *Arabic*: `kaggle.com/datasets/haithemhermessi/sanad-dataset`
- *Japanese*: `github.com/SkelterLabsInc/JaQuAD`

5.2 Results

5.2.1 Optimization toward the Pareto Set

In this Section, we discuss the results of EPSE, MGDA, PDO and EPOS on the examples. An illustration can be found in Figure 5.2,5.2, 5.3, 5.4, 5.5, 5.6, 5.7.

Compared to the other three algorithms, EPSE approximates the Pareto set instead of iteratively refining a point toward it. We expect that increasing the variable K, which represents the number of exploration directions generated by each point in the algorithm's queue, will result in a denser set of points on the Pareto front. However, since the total number of explored points remains constant regardless of K, increasing K reduces the overall explored region. This effect can be easily observed in the figures. Additionally, the algorithm inverts the Hessian matrix to determine a tangent direction (see Proposition 2.13). This inversion is computationally expensive and does not scale linearly with the number of features. A more detailed discussion on matrix inversion can be found in Section 5.2.3.

MGDA and PDO are both algorithms that start from an initial point $x \in \mathbb{R}^n$ and iteratively refine it toward the Pareto set. The resulting solution is selected randomly and cannot be directly influenced by the decision-maker. Despite this similarity, the two algorithms rely on fundamentally different techniques. MGDA is a gradient-based method that follows the minimum-norm direction within the convex hull of the objective gradients. In contrast, PDO employs bilevel optimization, where the inner level seeks a Pareto-stationary point.

However, as discussed in Section 4, the implementation of PDO closely resembles that of a gradient-based algorithm, aligning with the core idea of MGDA. The key distinction lies in how the descent direction is computed: MGDA approximates the solution by solving a linear system involving the Gram matrix rather than explicitly finding the minimum-norm vector in the convex hull of the gradients. Consequently, the two algorithms exhibit distinct trajectories toward the Pareto front.

EPOS successfully finds the preference-optimal solutions, even when the solutions lie at the extreme ends of the Pareto front. However, the optimization path does not necessarily follow the shortest trajectory from the initial starting point to the preference-optimal solution. Like MGDA, EPOS is also a gradient-based method, but it incorporates an additional linear program to determine not only the best descent direction but also a direction that moves toward a solution satisfying the given preferences. This approach can result in "knees," as visible in the figures.

Across all three sandbox settings, the algorithms successfully converge to-

ward the Pareto set. As expected, the only assumption required is continuous differentiability, which is satisfied by all three examples. Additionally, the algorithms guarantee convergence to a locally Pareto-optimal point, which in these cases aligns with the global Pareto-optimal solutions.

5.2.2 Example of Non-Convergence for PNGD

We repeated the non-convergence experiment for PNGD on Example 2.16, which is theoretically proven in Roy et al. (2023). As illustrated in Figure 5.8, PMM converges to the preference-optimal point within 10 iterations, whereas PNGD tends to converge toward $x = e_1$.

The issue of divergence in PNGD arises from its update rule: If the current point is close to the Pareto set, the algorithm fully updates f_0; otherwise, it optimizes toward the Pareto set aligned with the gradient of the criterion function. This behavior is evident in Figure 5.8(b). But as Roy et al. (2023) shows, it is not strong enough as the stationary point is bounded away from 0 converging to e_1 by this update rule.

5.2.3 Runtime of PMM

As shown in Figure 5.9, the runtime does not scale linearly with the feature size. One potential source of computational overhead in the algorithm is the inversion of a Hessian matrix. As the feature size increases, the size of the Hessians also grows. For a feature size of n, we have:

$$\nabla^2 f_{\hat{\beta}}(x) \in \mathbb{R}^{n \times n}. \tag{5.15}$$

Computing the inverse of such a matrix is computationally expensive. Conventional algorithms require $O(n^3)$ operations, while more advanced methods, such as Strassen's algorithm, achieve a complexity of $O(n^{\log_2 7})$ (Ikenmeyer and Lysikov 2019, Tonks 2019). Since matrix inversion is performed at each iteration, the overall algorithm cannot achieve linear runtime.

5.2.4 Optimization in the Pareto set for Model Merging

We performed a grid search for different coefficients $c \in \{0.1, 0.2, \ldots, 0.9\}^2$, similar to MAP. For each setting, we obtained the best-fitting parameters e_n, b_n, A_n for $n \in \{1, 2, 3\}$, corresponding to the approximations for the French, Arabic and Japanese metrics. The results are as follows:

$$A_{\text{French}} = \begin{bmatrix} 1.25 & -0.01 \\ -0.01 & 0.03 \end{bmatrix}, \quad b_{\text{French}} = \begin{bmatrix} -0.50 \\ 0.09 \end{bmatrix}, \quad e_{\text{French}} = 13.94 \tag{5.16}$$

$$A_{\text{Arabic}} = \begin{bmatrix} 1.24 & 0.15 \\ 0.15 & 0.11 \end{bmatrix}, \quad b_{\text{Arabic}} = \begin{bmatrix} -2.15 \\ -0.49 \end{bmatrix}, \quad e_{\text{Arabic}} = 21.52 \tag{5.17}$$

$$A_{\text{Japanese}} = \begin{bmatrix} 6.67 & -0.10 \\ -0.10 & 0.30 \end{bmatrix}, \quad b_{\text{Japanese}} = \begin{bmatrix} -4.59 \\ -0.05 \end{bmatrix}, \quad e_{\text{Japanese}} = 47.62 \tag{5.18}$$

Using these approximations, we searched for the preference optimal $c \in \mathbb{R}^2$, where

$$
\begin{aligned}
f_0 &= e_{\text{Japanese}} + b_{\text{Japanese}}^\top c + c^\top A_{\text{Japanese}} c \\
f_1 &= e_{\text{French}} + b_{\text{French}}^\top c + c^\top A_{\text{French}} c \\
f_2 &= e_{\text{Arabic}} + b_{\text{Arabic}}^\top c + c^\top A_{\text{Arabic}} c
\end{aligned} \tag{5.19}
$$

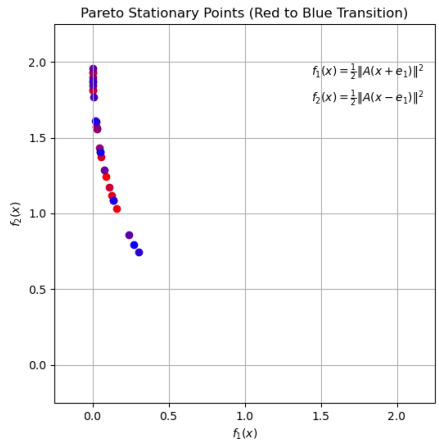
PMM results in $c = (0.68, 0.44)$ as illustrated in Figure 5.10. The final perplexity values are listed in Table 5.1.

	Base Model	**Fine-tuned Model**	**Merged Model**
Perplexity French	9.94	8.17	9.89
Perplexity Arabic	16.05	12.80	15.07
Perplexity Japanese	53.09	-	50.94

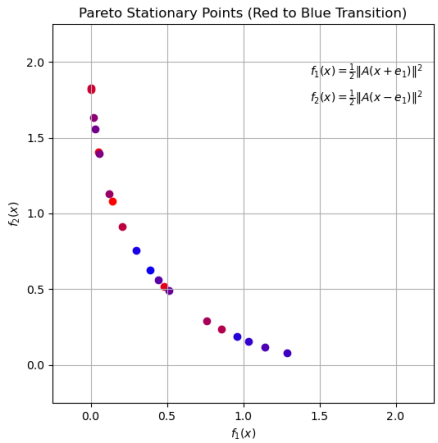
Table 5.1: Perplexity scores

This experiment requires a substantial amount of computational power, as we perform a grid search over possible coefficients c. Each evaluation involves computing the metrics for the merged models weighted by these coefficients. The larger the grid, the more accurately the parameters A_n, b_n, e_n can be estimated. Additionally, the size of the language dataset influences the accuracy of the perplexity values.

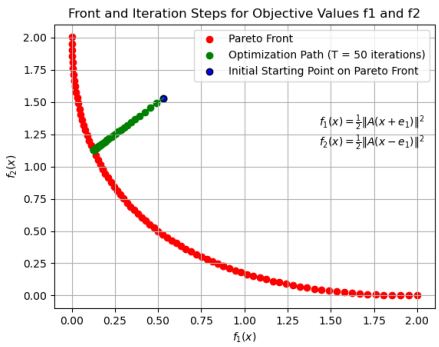
To conserve computational resources, we restricted the grid search to a smaller range and simplified the metric computation by considering at most 1000 lines of language text.



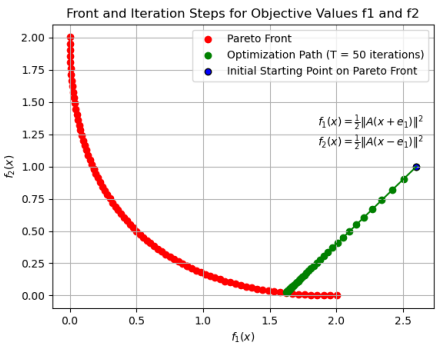
(a) EPSE: $K = 10$



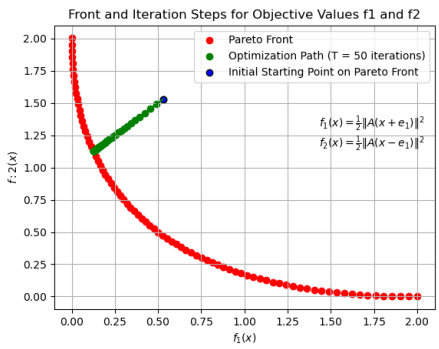
(b) EPSE: $K = 2$



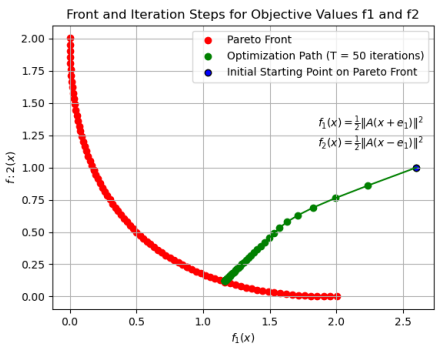
(c) MGDA: Initial Point x_1



(d) MGDA: Initial Point x_2

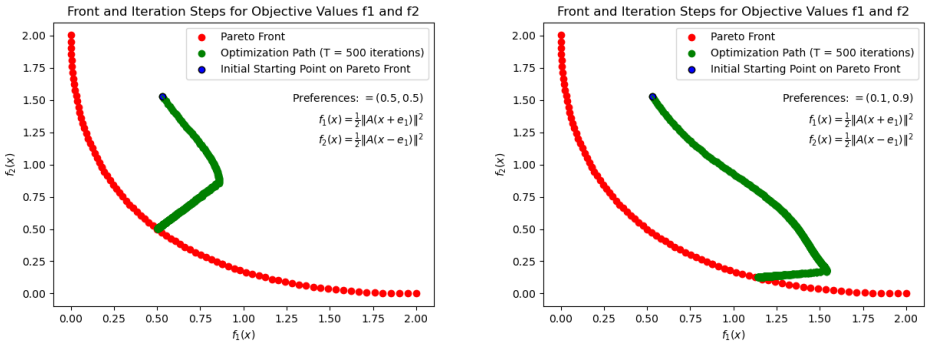


(e) PDO: Initial Point x_1



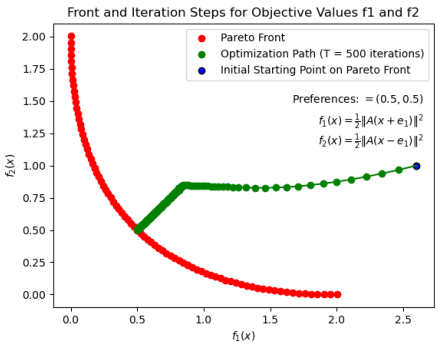
(f) PDO: Initial Point x_2

Figure 5.2: Illustration of EPSE, MGDA, PDO on Example 2.15

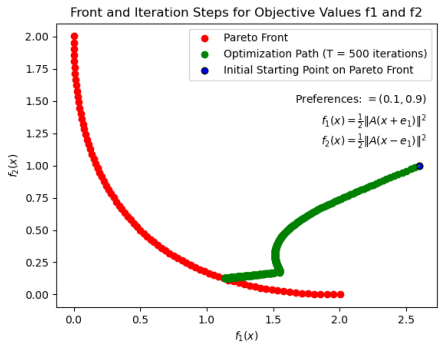


(a) EPOS with Init: x_1, Pref: l_1

(b) EPOS with Init: x_1, Pref: l_2

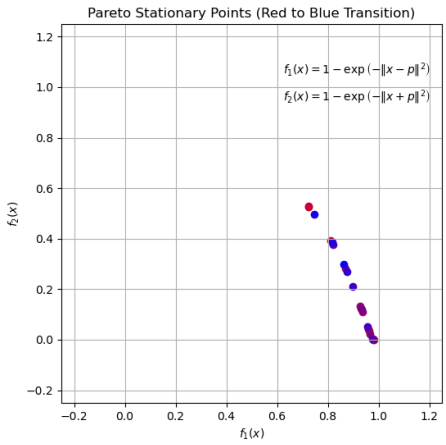


(c) EPOS with Init: x_2, Pref: l_1

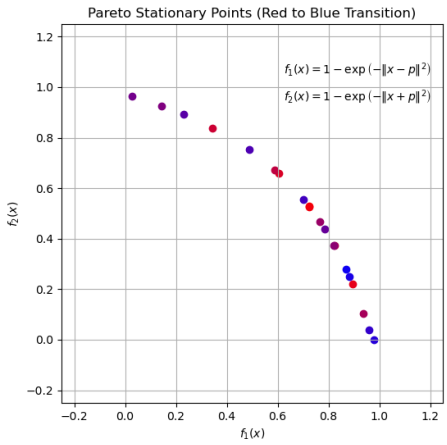


(d) EPOS with Init: x_2, Pref: l_2

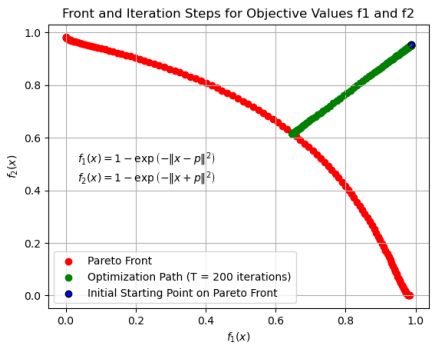
Figure 5.3: Illustration of EPOS on Example 2.15



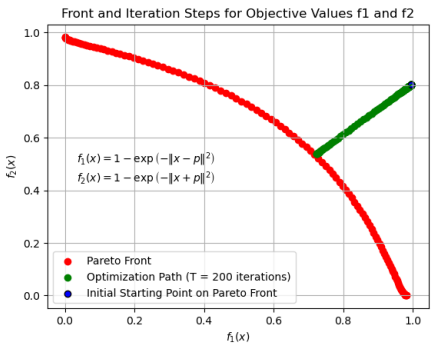
(a) EPSE: $K = 10$



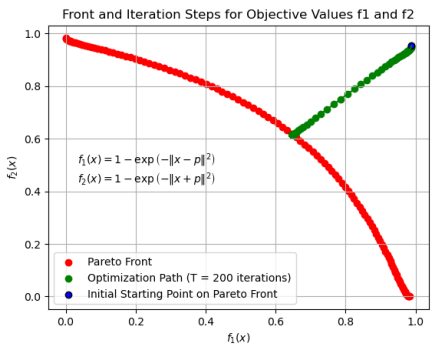
(b) EPSE: $K = 2$



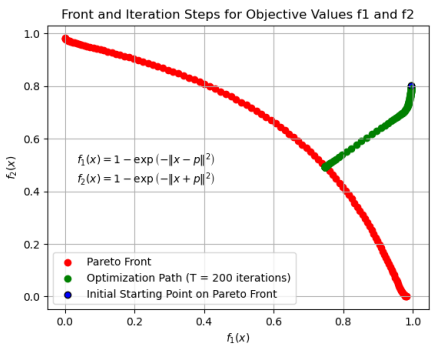
(c) MGDA: Initial Point x_1



(d) MGDA: Initial Point x_2

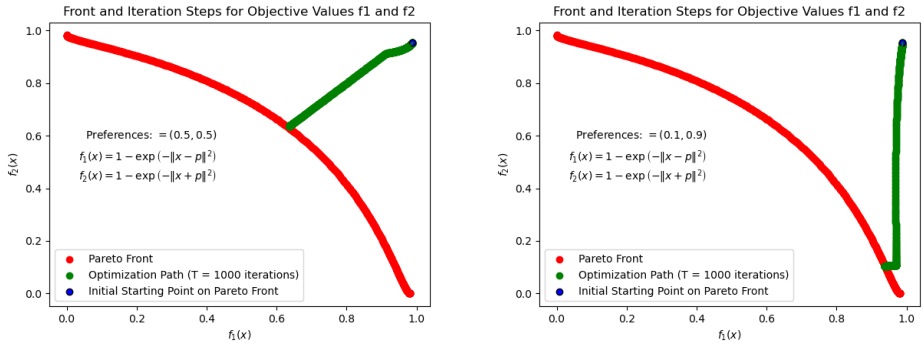


(e) PDO: Initial Point x_1



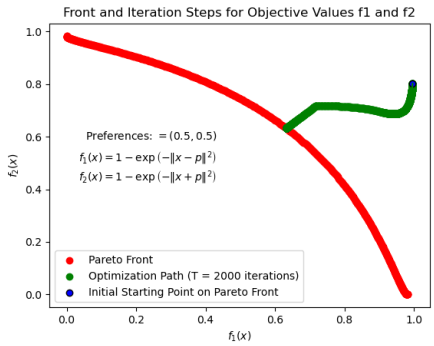
(f) PDO: Initial Point x_2

Figure 5.4: Illustration of EPSE, MGDA, PDO on Example 5.1

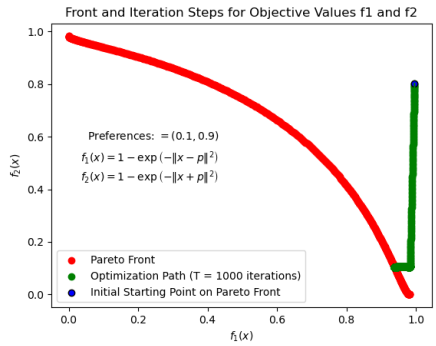


(a) EPOS with Init: x_1, Pref: l_1

(b) EPOS with Init: x_1, Pref: l_2

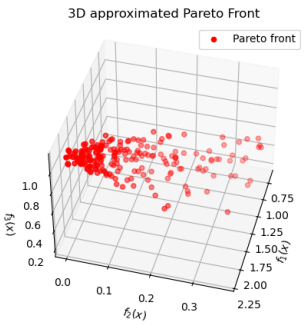


(c) EPOS with Init: x_2, Pref: l_1

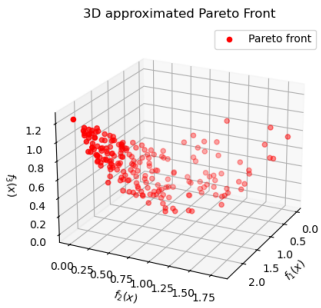


(d) EPOS with Init: x_2, Pref: l_2

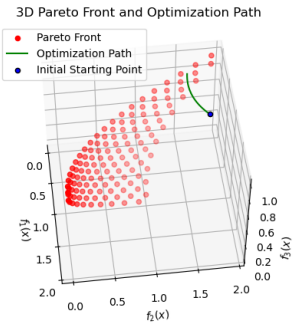
Figure 5.5: Illustration of EPOS on Example 5.1



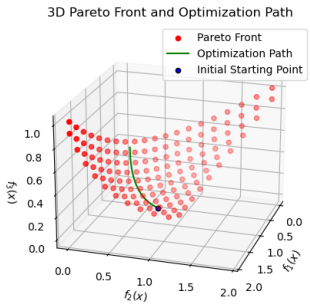
(a) EPSE: $K = 50$



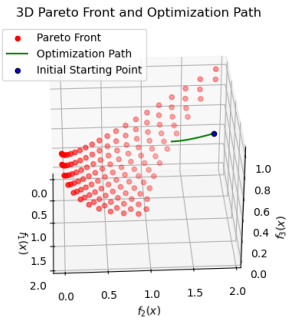
(b) EPSE: $K = 2$



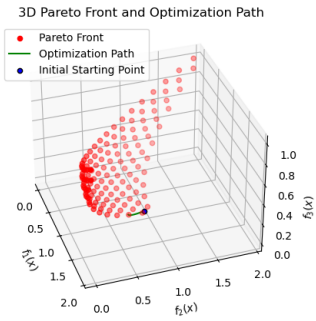
(c) MGDA: Initial Point x_1



(d) MGDA: Initial Point x_2

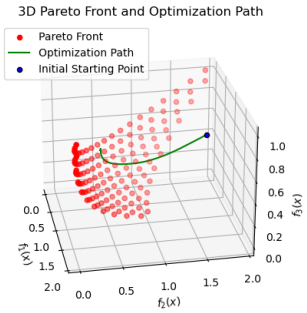


(e) PDO: Initial Point x_1

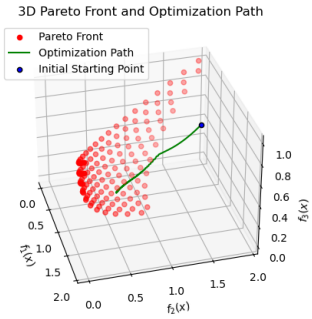


(f) PDO: Initial Point x_2

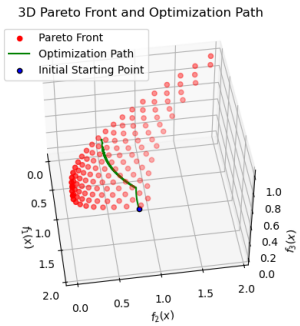
Figure 5.6: Illustration of EPSE, MGDA and PDO on Example 5.2



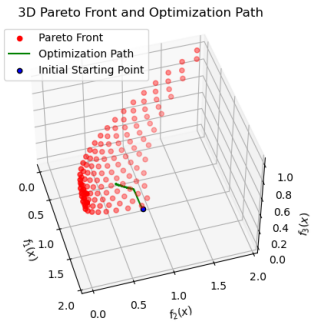
(a) EPOS with Init: x_1, Pref: l_1



(b) EPOS with Init: x_1, Pref: l_2

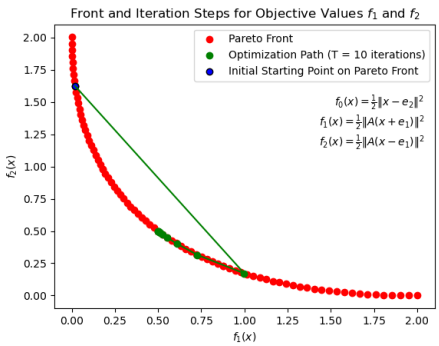


(c) EPOS with Init: x_2, Pref: l_1

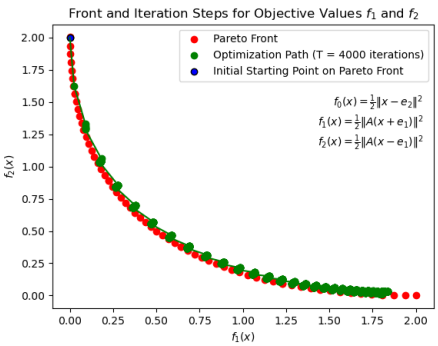


(d) EPOS with Init: x_2, Pref: l_2

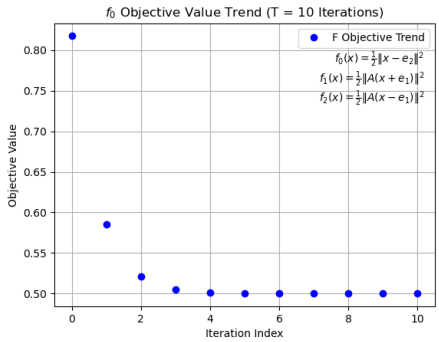
Figure 5.7: Illustration of EPOS on Example 5.2



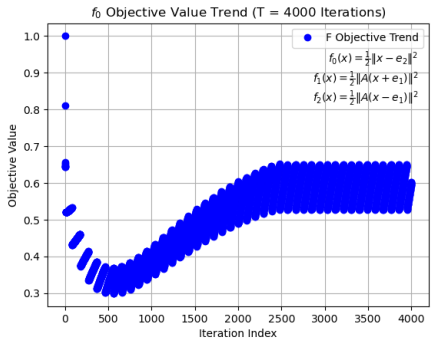
(a) PMM: Convergence to F_{pref}



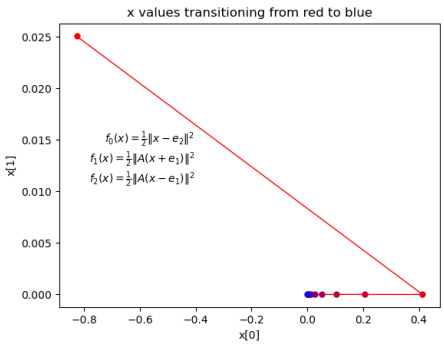
(b) PNGD: Divergence from F_{pref}



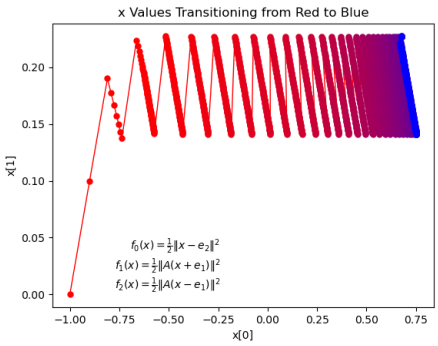
(c) PMM: f_0 trend



(d) PNGD: f_0 trend

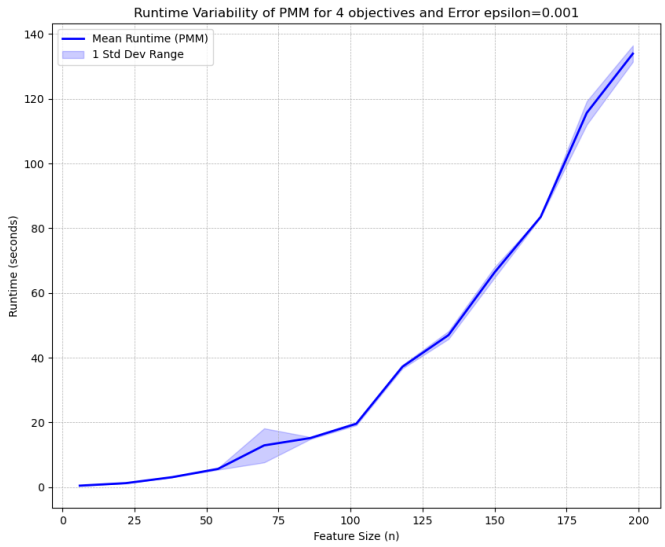


(e) PMM: x-path

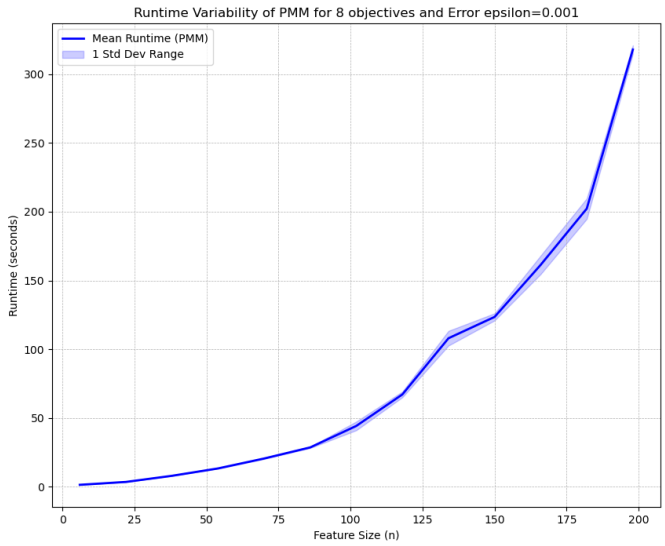


(f) PNGD: x-path

Figure 5.8: Illustration of PNGD and PMM on example 2.16

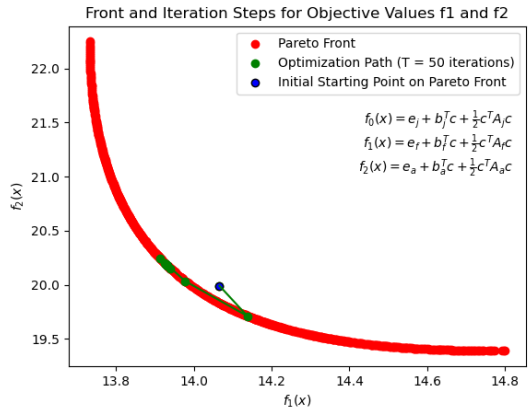


(a) for 4 objective functions

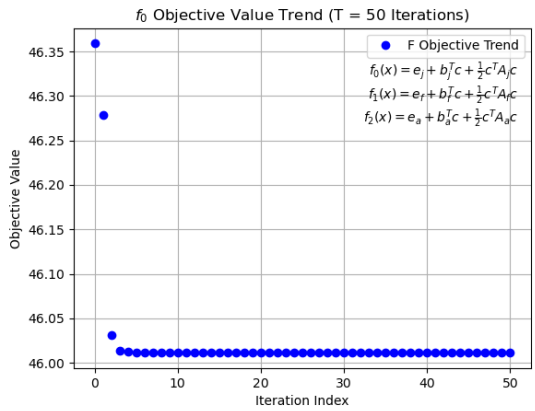


(b) for 8 objective functions

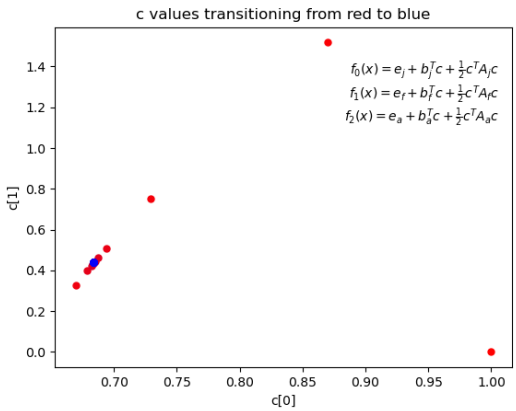
Figure 5.9: Runtime depending on feature size



(a) Convergence illustrated on Pareto front



(b) Improvement of f_0



(c) c-Path in $\mathbb{R}^2$

Figure 5.10: Convergence of PMM in Model Merging

Chapter 6

Conclusion

In this work, our goal was to understand optimization within the Pareto set. To achieve this, we first focused on optimization toward the Pareto set and analyzed four of the most recent algorithms from different optimization categories, including representative subsampling, bilevel optimization, a gradient-based algorithm without user preferences, and a gradient-based algorithm with user preferences. We evaluated their performance on various examples, including two convex objectives, two concave objectives, and three convex objectives.

Our analysis revealed that the way bilevel optimization is implemented closely aligns with a gradient-based algorithm that finds the minimum-norm vector in the convex hull of the objective gradients. We found that subsampling is computationally expensive, as the Pareto set is typically infinite, and moving along a tangent direction requires inverting the Hessian. Preference-based gradient-based algorithms converge if the necessary assumptions are met but only consider preferences as weights for the objectives. Notably, no preference-based algorithm exists that optimizes an additional criterion function.

Currently, only two algorithms exist that aim to optimize within the Pareto set for an additional criterion function. PNGD, the earlier of the two, follows a gradient-descent approach intended to iteratively approach the preference-optimal solution. Roy et al. (2023) theoretically proved that PNGD does not converge to the preference-optimal solution, a result that we verified experimentally. As PMM is the most recent algorithm that guarantees convergence, we examined its runtime behavior as a function of the feature size. Our observations indicate that it does not scale linearly due to the inversion of Hessians, which runs in at least $O(n^{\log_2 7})$.

We explored a use case for optimization within the Pareto set in the context of model merging. Specifically, we extended the MAP algorithm, which

approximates the Pareto set but does not incorporate user preferences. Using fine-tuned French and Arabic language models, we aimed to achieve Pareto-optimal merged models that balance performance on French and Arabic metrics while optimizing for Japanese as a preference. This approach presents an interesting direction for further research and potential improvements.

Bibliography

Silvere Bonnabel. *Stochastic Gradient Descent on Riemannian Manifolds*, volume 58. IEEE, 2013.

Stephen P Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.

Yair Censor. Pareto optimality in multiobjective problems. *Applied Mathematics and Optimization*, 4(1):41–59, 1977.

Haotian Cui, Chloe Wang, Hassaan Maan, Kuan Pang, Fengning Luo, Nan Duan, and Bo Wang. scgpt: toward building a foundation model for single-cell multi-omics using generative ai. *Nature Methods*, 21(8):1470–1480, 2024.

Emiliano De Cristofaro. An overview of privacy in machine learning. *arXiv preprint arXiv:2005.08679*, 2020.

Jean-Antoine Désidéri. Multiple-gradient descent algorithm (mgda) for multiobjective optimization. *Comptes Rendus Mathematique*, 350(5-6):313–318, 2012.

Joerg Fliege, LM Grana Drummond, and Benar Fux Svaiter. Newton's method for multiobjective optimization. *SIAM Journal on Optimization*, 20(2):602–626, 2009.

Jörg Fliege and Benar Fux Svaiter. Steepest descent methods for multicriteria optimization. *Mathematical methods of operations research*, 51:479–494, 2000.

Shaona Ghosh, Chris Lovell, and Steve R Gunn. Towards pareto descent directions in sampling experts for multiple tasks in an on-line learning paradigm. In *AAAI Spring Symposium: Lifelong Machine Learning*, 2013.

Leilani H Gilpin, David Bau, Ben Z Yuan, Ayesha Bajwa, Michael Specter, and Lalana Kagal. Explaining explanations: An overview of interpretability of machine learning. In *2018 IEEE 5th International Conference on data science and advanced analytics (DSAA)*, pages 80–89. IEEE, 2018.

Alfred Göpfert and Reinhard Nehse. Vektoroptimierung: Theorie, verfahren und anwendungen. *(No Title)*, 1990.

Claus Hillermeier. Generalized homotopy approach to multiobjective optimization. *Journal of Optimization Theory and Applications*, 110(3):557–583, 2001.

Christian Ikenmeyer and Vladimir Lysikov. Strassen's $2\times$ 2 matrix multiplication algorithm: a conceptual perspective. *ANNALI DELL'UNIVERSITA'DI FERRARA*, 65(2):241–248, 2019.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

Ken Kobayashi, Naoki Hamada, Akiyoshi Sannai, Akinori Tanaka, Kenichi Bannai, and Masashi Sugiyama. Bézier simplex fitting: Describing pareto fronts of simplicial problems with small samples in multi-objective optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 2304–2313, 2019.

Aditya Kulkarni, Maximilian Kohns, Michael Bortz, Karl-Heinz Küfer, and Hans Hasse. Regularities of pareto sets in low-dimensional practical multi-criteria optimisation problems: analysis, explanation, and exploitation. *Optimization and Engineering*, 24(3):1611–1632, 2023.

Xi Lin, Hui-Ling Zhen, Zhenhua Li, Qing-Fu Zhang, and Sam Kwong. Pareto multi-task learning. *Advances in neural information processing systems*, 32, 2019.

Zhiqi Bu Lu Li, Tianyu Zhang and Suyuchen Wang. Map: Low-compute model merging with amortized pareto fronts via quadratic approximation. 2024.

D. T. Luc. *Theory of Vector Optimization*. Springer-Verlag, Berlin, 1988.

Pingchuan Ma, Tao Du, and Wojciech Matusik. Efficient continuous pareto exploration in multi-task learning. In *International Conference on Machine Learning*, pages 6522–6531. PMLR, 2020.

Debabrata Mahapatra and Vaibhav Rajan. Multi-task learning with user preferences: Gradient descent with controlled ascent in pareto optimization. In *International Conference on Machine Learning*, pages 6597–6607. PMLR, 2020.

Mohammad Mahdi Kamani, Rana Forsati, James Z Wang, and Mehrdad Mahdavi. Pareto efficient fairness in supervised learning: From extraction to tracing. *arXiv e-prints*, pages arXiv–2104, 2021.

I Maruşciac. On fritz john type optimality criterion in multi-objective optimization. *Mathematica-Revue d'analyse numérique et de théorie de l'approximation. L'analyse numérique et la théorie de l'approximation*, pages 109–114, 1982.

Yurii Nesterov. *Introductory lectures on convex optimization: A basic course*, volume 87. Springer Science & Business Media, 2013.

Andre Opris, Duc-Cuong Dang, Frank Neumann, and Dirk Sudholt. Runtime analyses of nsga-iii on many-objective problems. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 1596–1604, 2024.

Dana Pessach and Erez Shmueli. A review on fairness in machine learning. *ACM Computing Surveys (CSUR)*, 55(3):1–44, 2022.

Robi Polikar. Ensemble learning. *Ensemble machine learning: Methods and applications*, pages 1–34, 2012.

Abhishek Roy, Geelon So, and Yi-An Ma. Optimization on pareto sets: On a theory of multi-objective optimization. *arXiv preprint arXiv:2308.02145*, 2023.

Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Yalin E Sagduyu, Tugba Erpek, Aylin Yener, and Sennur Ulukus. Low-latency task-oriented communications with multi-round, multi-task deep learning. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, pages 2365–2370, 2024.

Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization. *Advances in neural information processing systems*, 31, 2018.

Hila Sheftel, Oren Shoval, Avi Mayo, and Uri Alon. The geometry of the p areto front in biological phenotype space. *Ecology and evolution*, 3(6): 1471–1483, 2013.

Mun-Bo Shim, Myung-Won Suh, Tomonari Furukawa, Genki Yagawa, and Shinobu Yoshimura. Pareto-based continuous evolutionary algorithms for multiobjective optimization. *Engineering Computations*, 19(1):22–48, 2002.

Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. Large language models in medicine. *Nature medicine*, 29(8):1930–1940, 2023.

Zak Tonks. On fast matrix inversion via fast matrix multiplication. *arXiv preprint arXiv:1901.00904*, 2019.

Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International conference on machine learning*, pages 23965–23998. PMLR, 2022.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin A Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.

Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. Adamerging: Adaptive model merging for multi-task learning. *arXiv preprint arXiv:2310.02575*, 2023.

Mao Ye and Qiang Liu. Pareto navigation gradient descent: a first-order algorithm for optimization in pareto set. In *Uncertainty in Artificial Intelligence*, pages 2246–2255. PMLR, 2022.

Yuan Yuan, Hua Xu, and Bo Wang. An improved nsga-iii procedure for evolutionary many-objective optimization. In *Proceedings of the 2014 annual conference on genetic and evolutionary computation*, pages 661–668, 2014.

Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 11975–11986, 2023.